Duke | PRATT SCHOOL of ENGINEERING

CYBERSEC 590

AI in Modern Security Operations

WEEK 2:
AGENT FUNDAEMNTALS

OVERVIEW

Reading quiz

Slido, on Building Effective Agents and the AISI post

LLM mechanics

Tokens, the context window, sampling parameters

From LLM to agent

What a tool is and how a tool call actually executes

Orchestration patterns

Five workflow patterns plus autonomous agents

Housekeeping



Assignment 1 due Sep 9



Reading quiz

Counts toward class participation.



Assignment 2: Next week

Assignment will be given every other week (6 in total)

WORKFLOWS AND AGENTS ARE BOTH 'AGENTIC SYSTEMS'

WORKFLOW

LLMS AND TOOLS ARE ORCHESTRATED THROUGH PREDEFINED PATHS.

Your **code** decides the route. The model fills in the steps.

AGENT

LLMS DYNAMICALLY DIRECT THEIR OWN PROCESSES AND TOOL USAGE.

The model decides the route, based on what it observes.

IN PRACTICE THIS IS A SPECTRUM



your code decides

the model decides

“START WITH SIMPLE PROMPTS, OPTIMIZE THEM WITH COMPREHENSIVE EVALUATION, AND ADD MULTI-STEP AGENTIC SYSTEMS ONLY WHEN SIMPLER SOLUTIONS FALL SHORT.”

-Building Effective Agents

Single model call

one call, with retrieval and well-chosen context

Workflow patterns

predefined paths: tonight's five

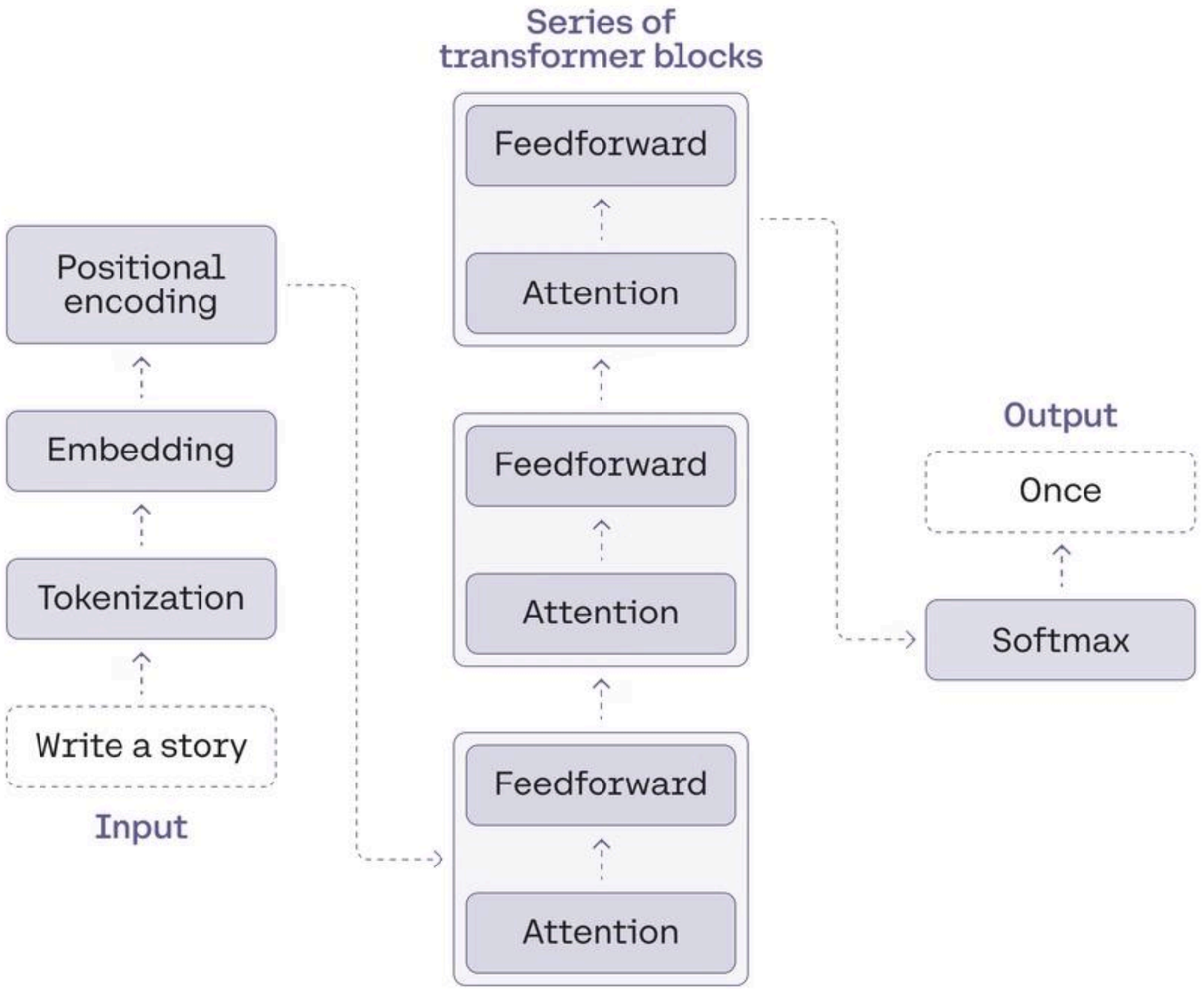
Autonomous agent

the model steers; you set budgets and gates

each step right adds: cost · latency · failure surface · debugging pain

LLMS
THE BRAINS BEHIND THE AGENTS

LLMS ARE NEXT TOKEN PREDICTORS



HOW TOKEN GENERATION WORKS

1	**Tokenization**	Input text → subword tokens
2	**Embeddings**	Tokens → dense vector representations
3	**Transformer Layers**	Attention + feed-forward processing
4	**Probability Distribution**	Final layer → prob over entire vocabulary
5	**Sampling Strategy**	Select next token from distribution
6	**Repeat**	Until stop condition

1. TOKENIZATION



1. TOKENIZATION

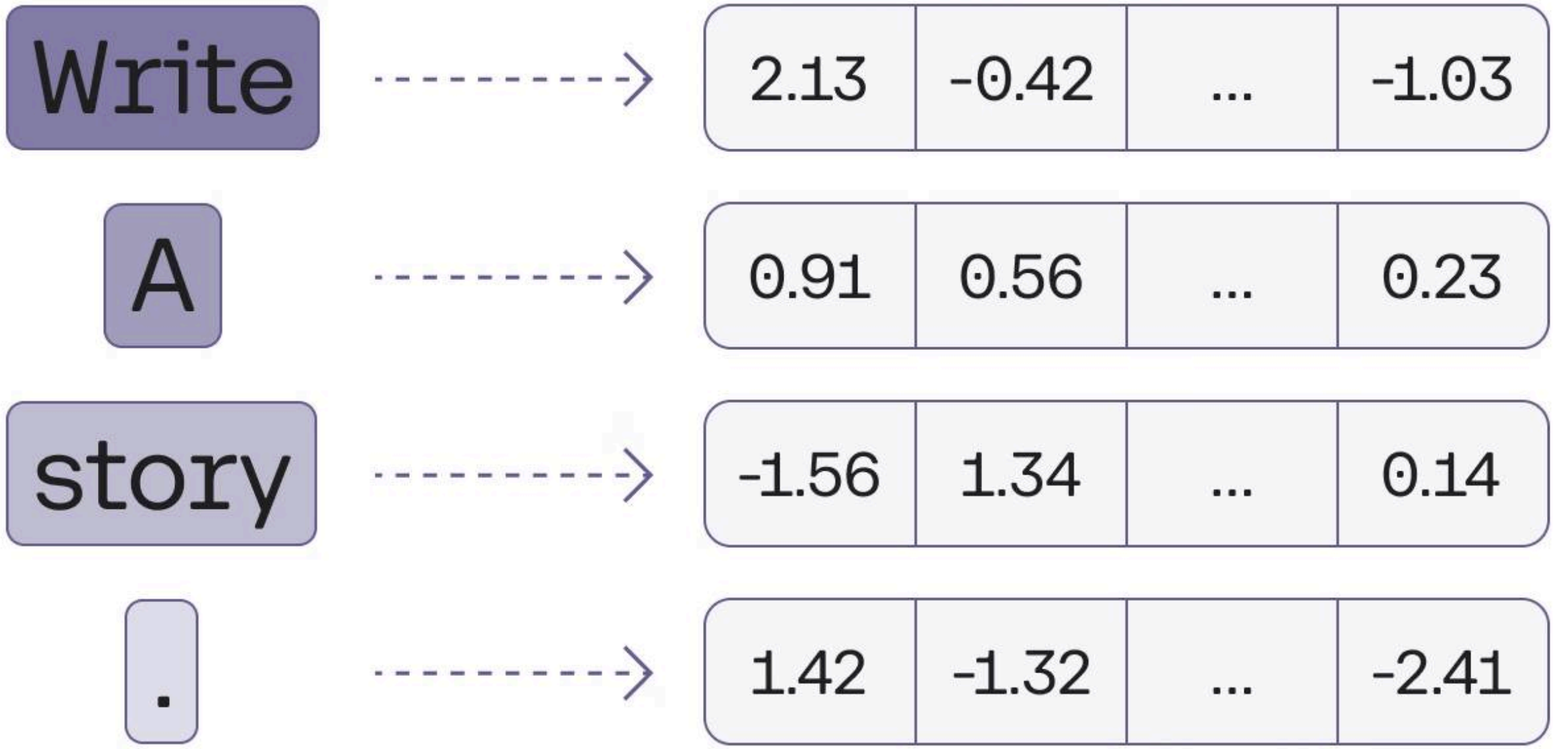
English

`Susp` `icious` `·Power` `Shell` `·activity` `·was` `·detected` `.`

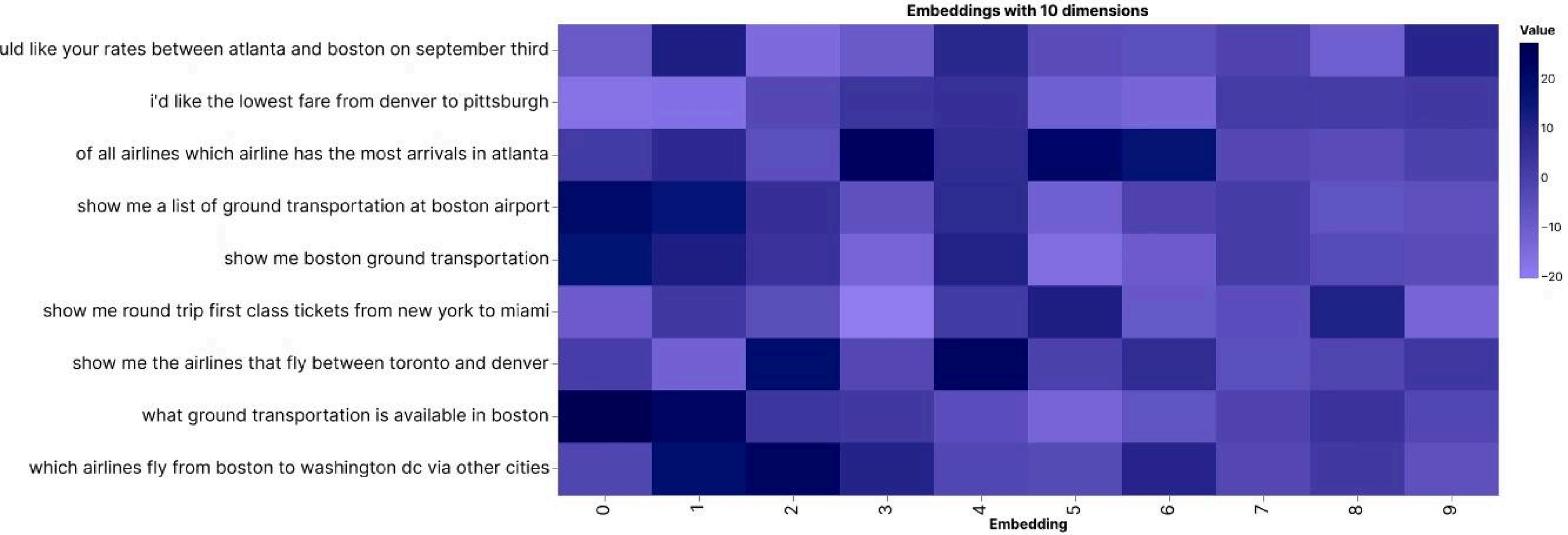
SHA-256 fragments

`e` `3` `b` `0` `c` `442` `98` `fc` `1` `c` `14` `9` `af` `bf` `4` `c` `89` `96` `fb`

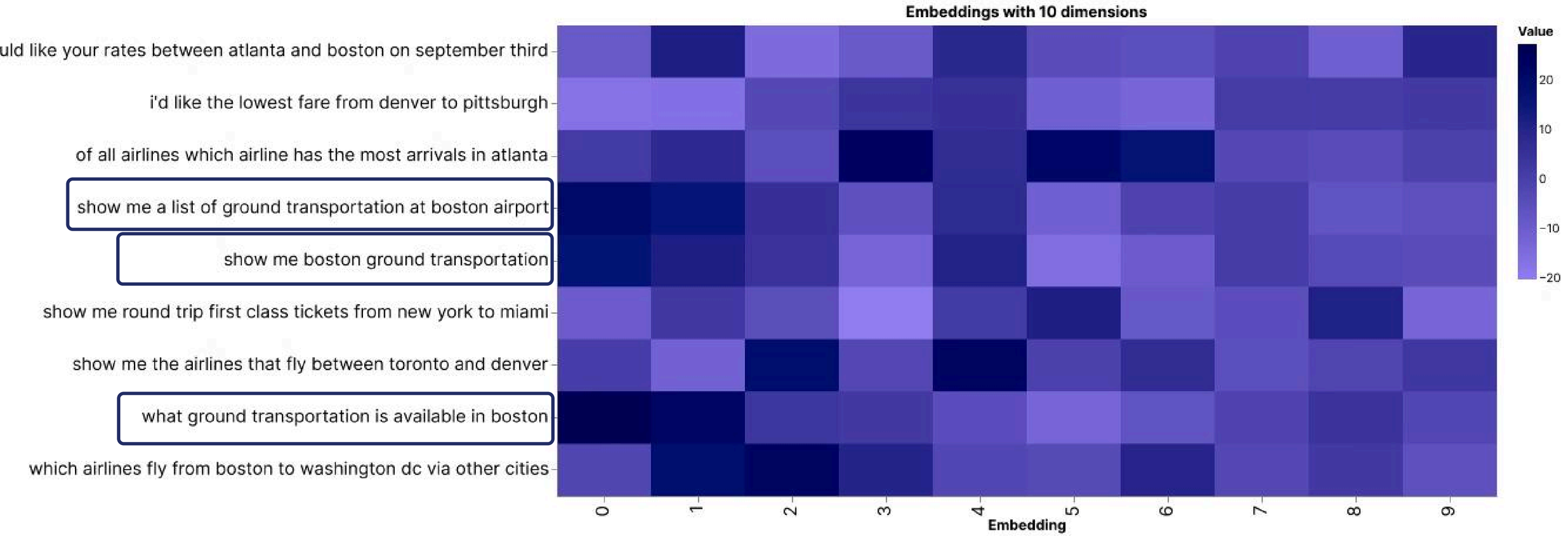
2. EMBEDDING



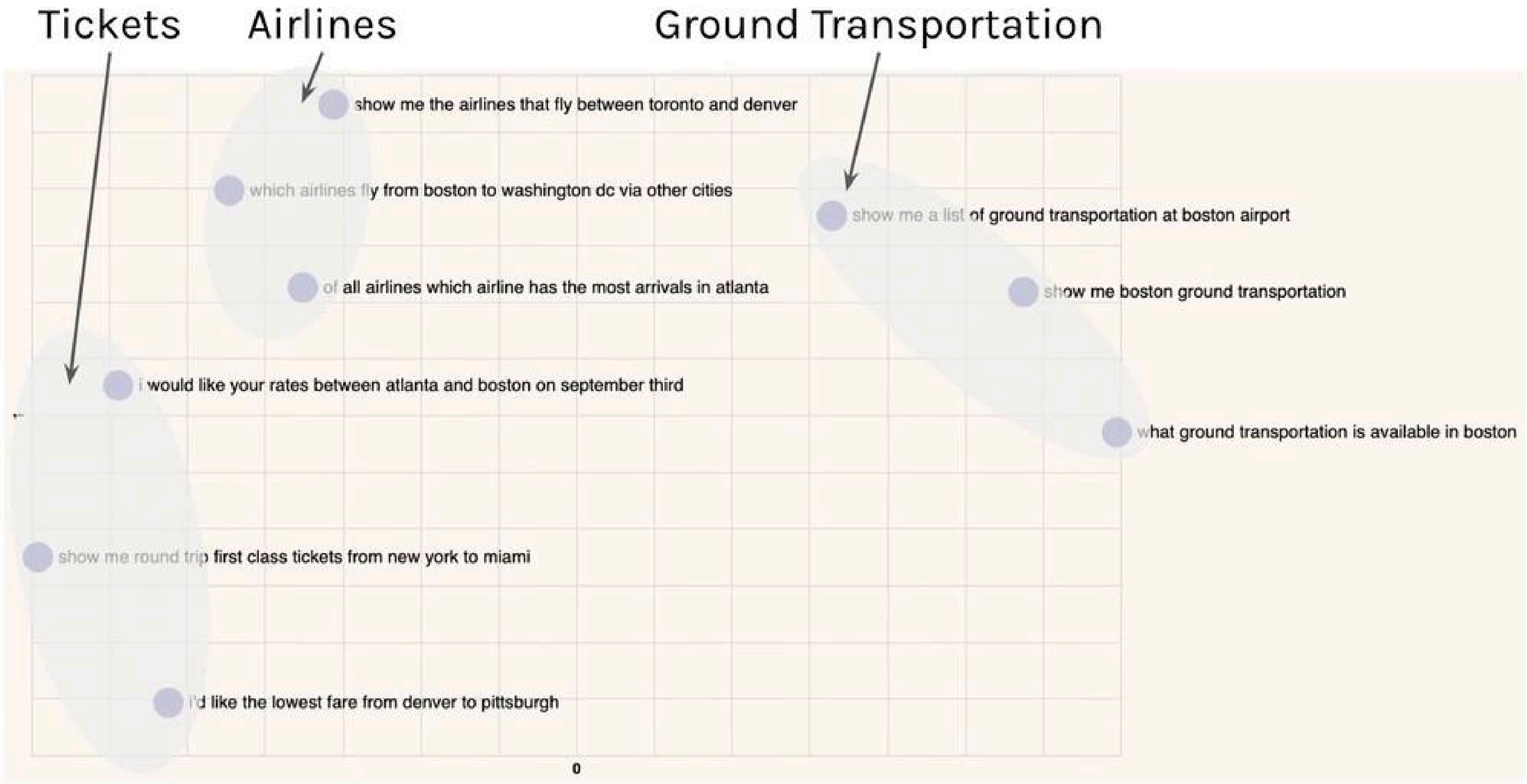
EMBEDDING DETECTS SIMILARITIES



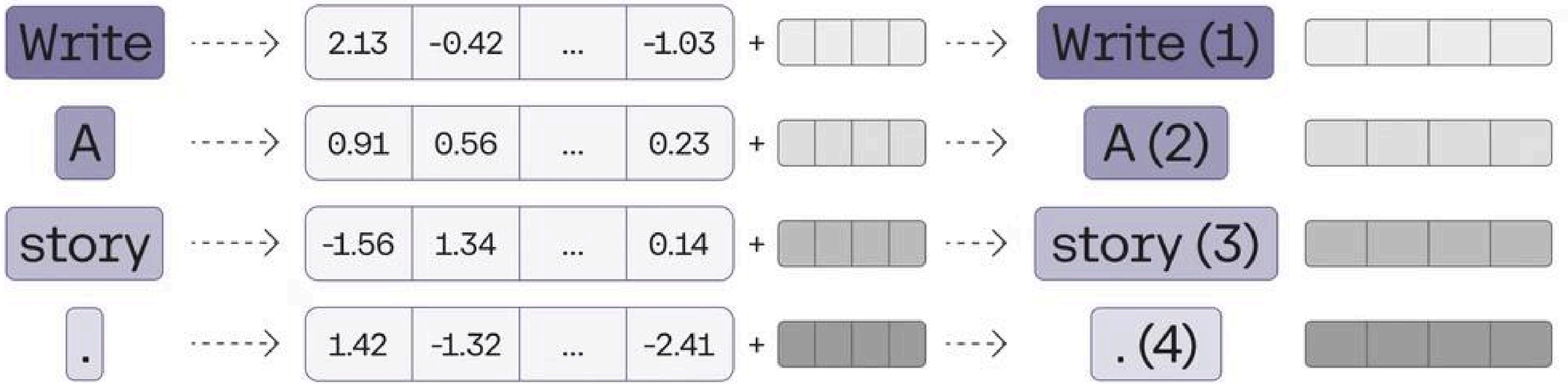
EMBEDDING DETECTS SIMILARITIES



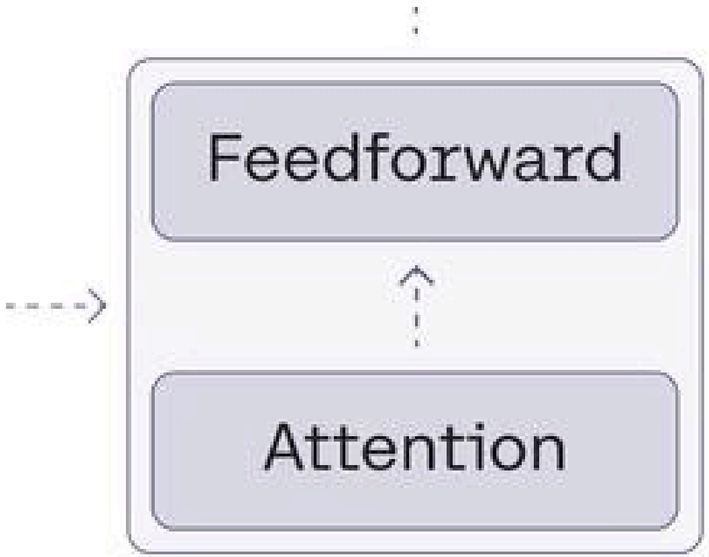
EMBEDDING DETECTS SIMILARITIES

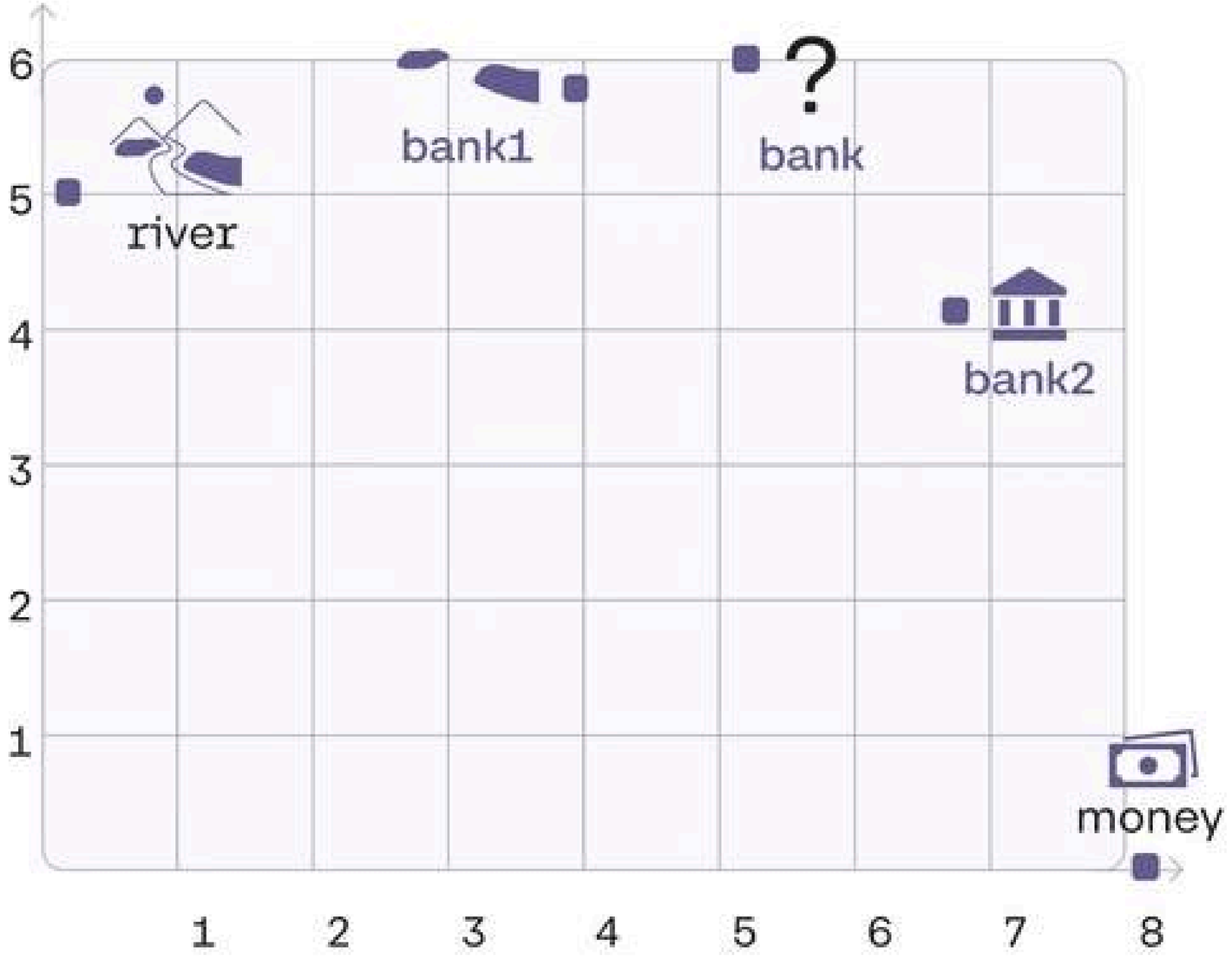


3. POSITIONAL ENCODING

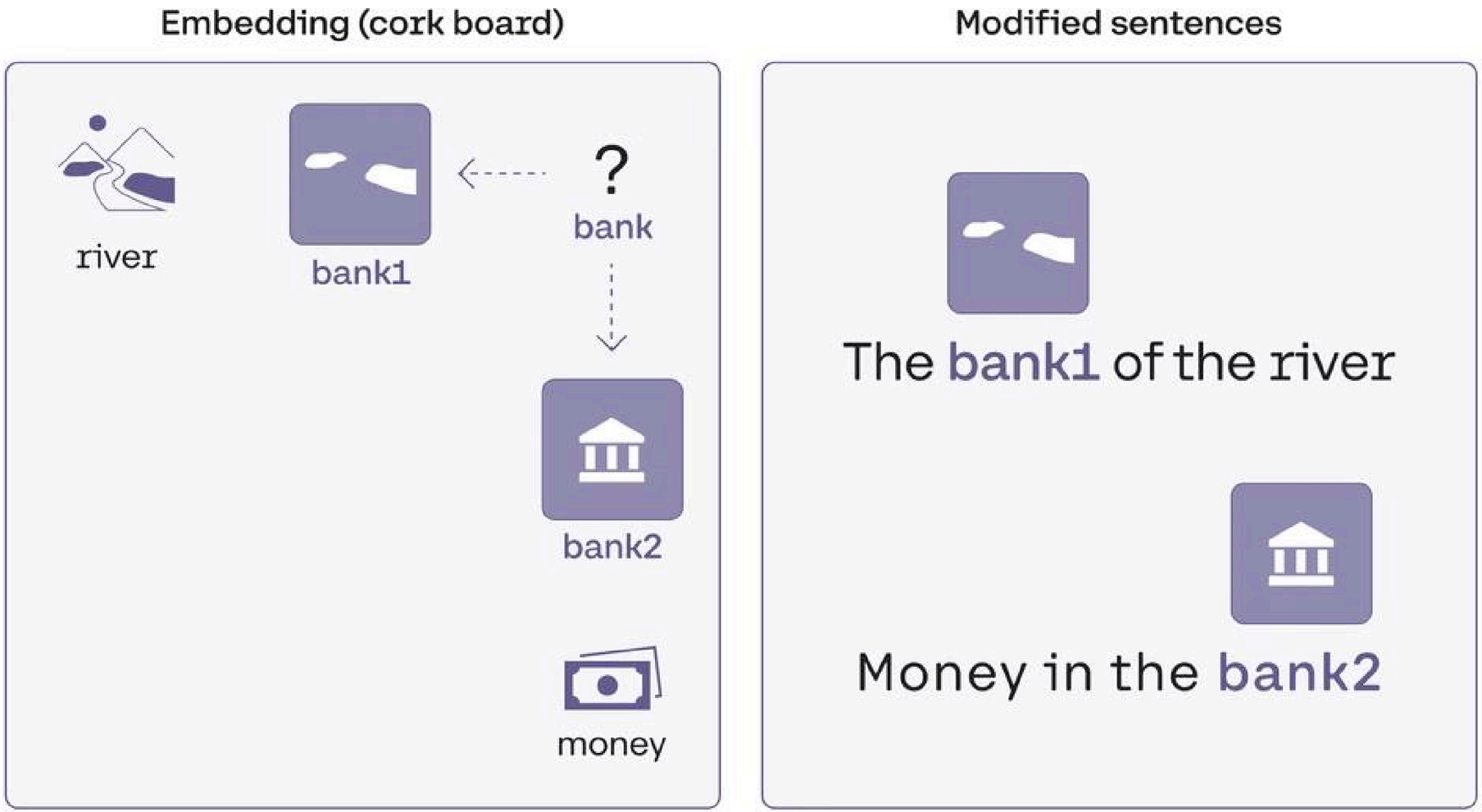


4. ATTENTION

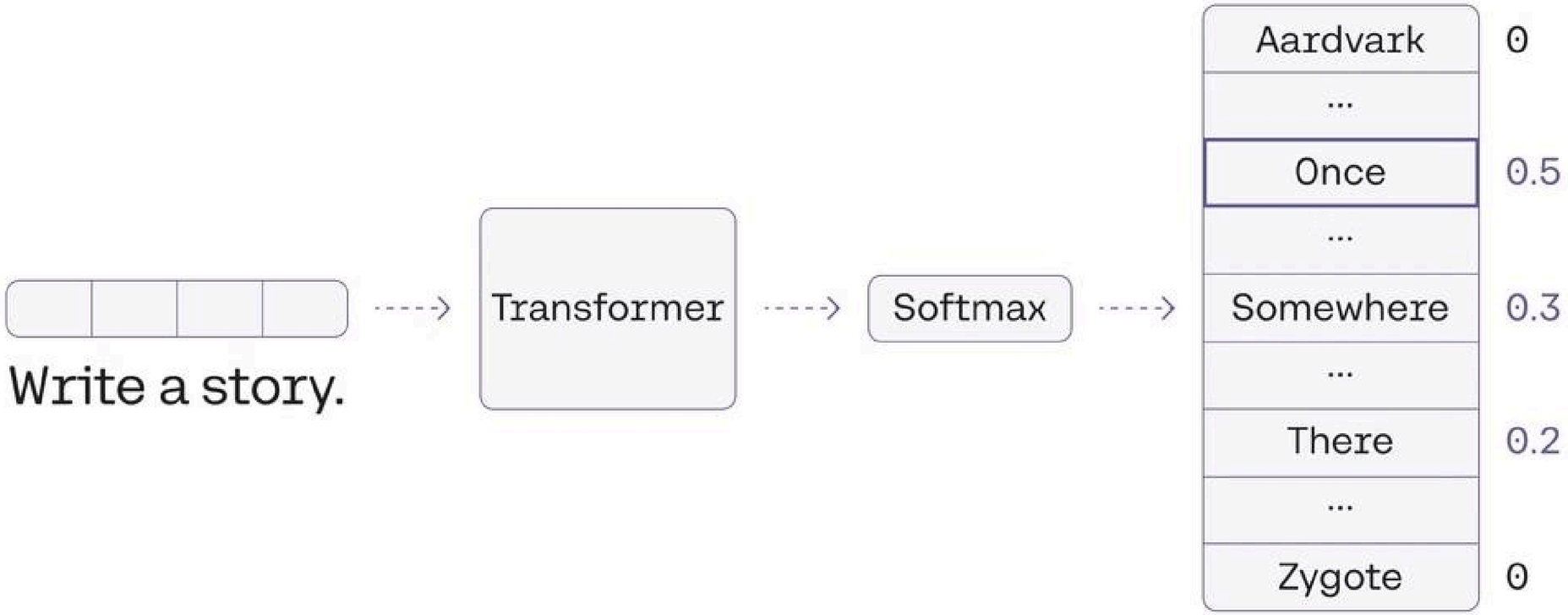



6
5
4
3
2
1
river
bank1
bank
?
bank2
money
1
2
3
4
5
6
7
8

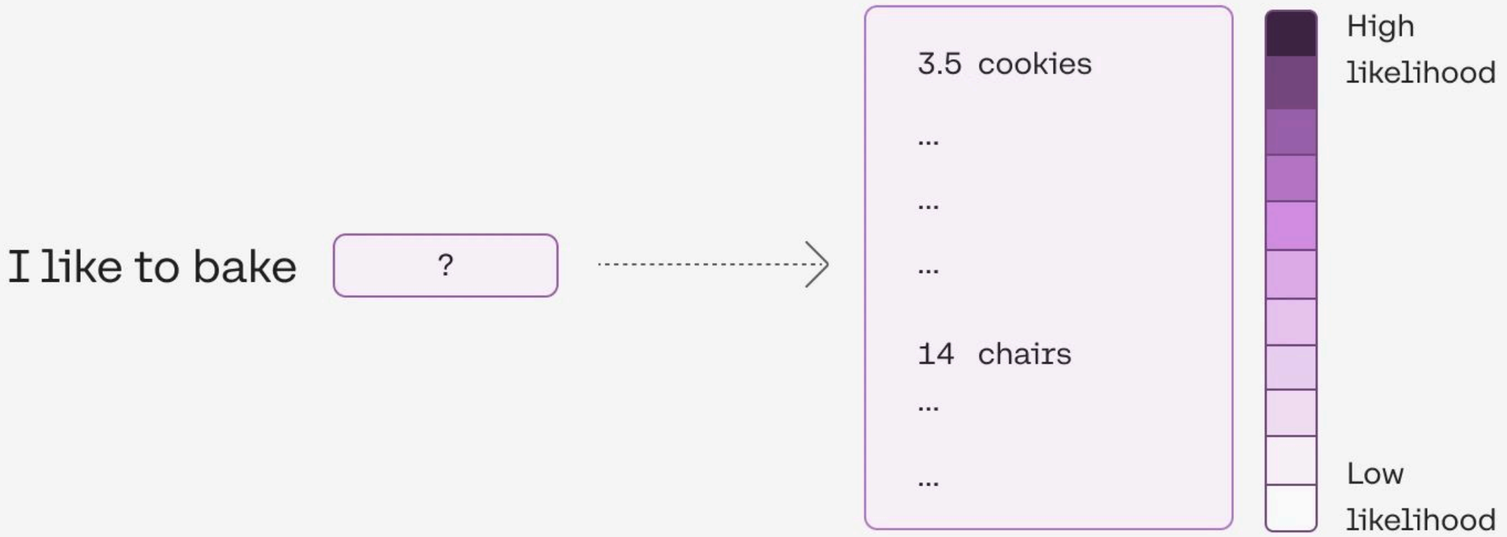
4. ATTENTION


Embedding (cork board)
river
bank1
?
bank
bank2
money
Modified sentences
The bank1 of the river
Money in the bank2

5. PROBABILITY DISTRIBUTION



6. SAMPLING STRATEGY



Top-k Sampling

Limits selection to the k most probable tokens, then renormalizes.

k = 1

Greedy decoding

Always pick the most probable token. Deterministic.

k = 50

Common default

Choose from top 50 tokens. Good general setting.

k = vocab_size

Full vocabulary

Consider all tokens. Equivalent to no top-k filtering.

The Problem with Fixed k

Fixed k doesn't adapt to context. Sometimes the model is very confident (top 3 tokens hold 95% probability) and sometimes it's uncertain (probability spread across hundreds).

Top-p (Nucleus Sampling)

Dynamically selects the smallest set of tokens whose cumulative probability exceeds p.

p = 0.1

Very focused — only highest probability tokens

p = 0.9

Wide selection — most of the distribution

p = 1.0

All tokens considered

Advantage over Top-k:

Adapts to model confidence. When the model is sure, few tokens are considered. When uncertain, more are explored.

HYPERPARAMETERS
TUNING THE LLMS

TEMPERATURE: CONTROLLING RANDOMNESS

$$p(\text{token}_i) = \exp(\text{logit}_i / T) / \Sigma \exp(\text{logit}_j / T)$$

$T \to 0$

Deterministic

Distribution becomes peaked at highest-probability token. Same input always produces same output.

$T = 1.0$

Raw Distribution

Unmodified model probabilities. The default — balanced between focus and exploration.

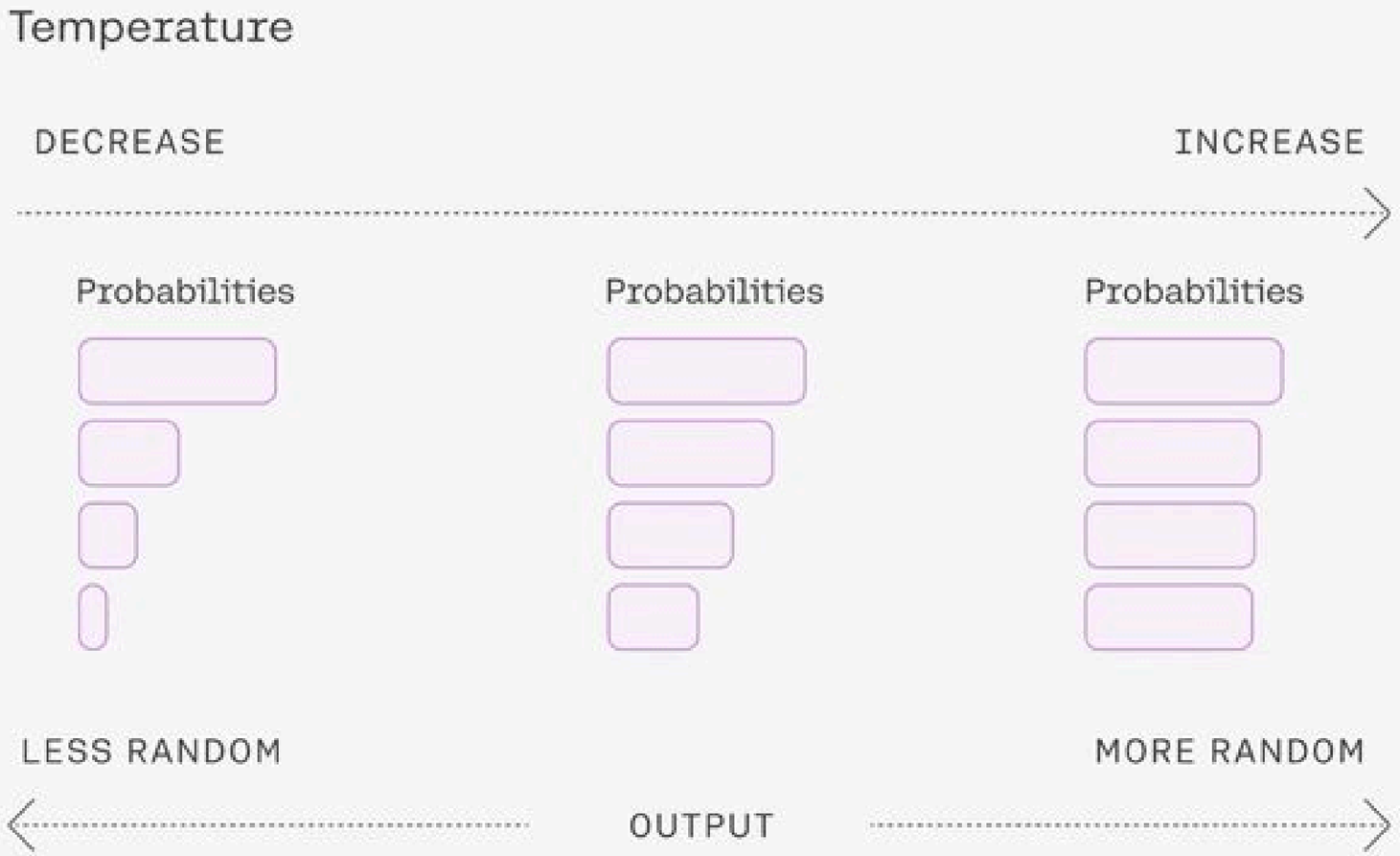
$T > 1.0$

Creative / Chaotic

Flattens distribution. More randomness, more surprising outputs. Risk of incoherence.

Physics analogy: Higher temperature = more molecular disorder = more randomness in which state the system lands in.

TEMPERATURE: CONTROLLING RANDOMNESS



TEMPERATURE: CONTROLLING RANDOMNESS

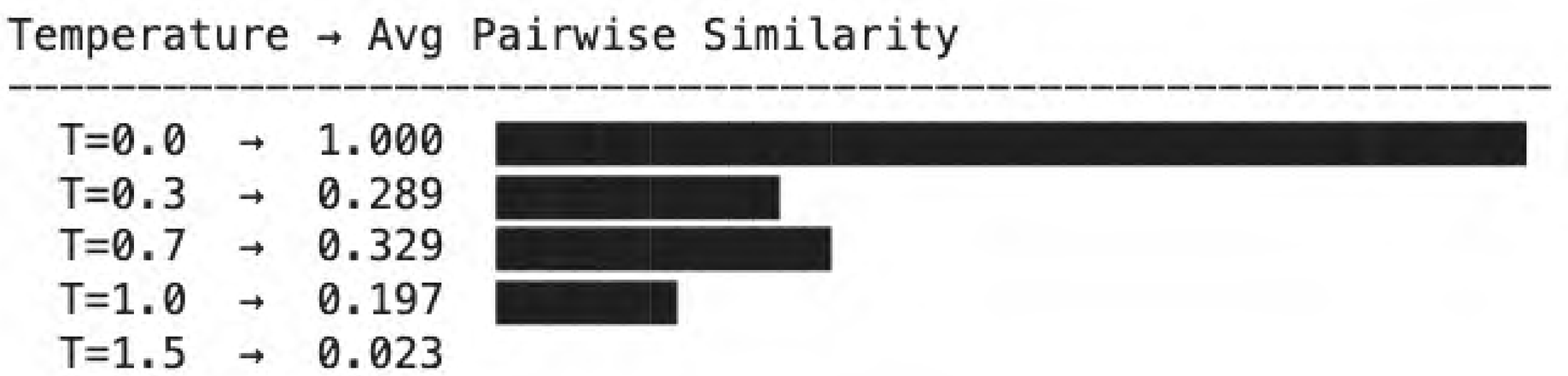
```
PROMPT = """Analyze this network log entry and classify as BENIGN or MALICIOUS:

SRC: 10.0.1.45  DST: 198.51.100.23  PROTO: TCP
PORT: 443  BYTES: 847293  DURATION: 3600s
DNS_QUERY: update-check.definitelynotmalware.xyz
INTERVAL: every 3600s ± 12s (24 observations)

Provide your classification and reasoning."""
```

Temp	Classification	Trial 1	Trial 2	Trial 3
0	MALICIOUS	Domain Name: The DNS query includes a suspicious domain name, "definitelynotmalware.xyz," whic	Domain Name: The DNS query includes a suspicious domain name, "definitelynotmalware.xyz," whic	Domain Name: The DNS query includes a suspicious domain name, "definitelynotmalware.xyz," whic
0.3	MALICIOUS	The destination IP (198.51.100.23) is a public IP, and the domain (definitelynotmalware.xyz) i	The destination IP (198.51.100.23) is a public IP, and the DNS query points to a domain with a	Domain Name: The DNS query includes a suspicious domain name, "definitelynotmalware.xyz," which
0.7	MALICIOUS	The destination IP (198.51.100.23) is associated with a domain (definitelynotmalware.xyz) that	Destination IP (198.51.100.23) is known to be associated with malicious activity. Domain (defin	The destination IP (198.51.100.23) is a public IP, and the DNS query resolves to a domain with
1	MALICIOUS	Domain Name: The DNS query is for update-check.definitelynotmalware.xyz, which has a misleadi	The destination IP (198.51.100.23) and domain (definitelynotmalware.xyz) are suspicious. The do	The destination IP (198.51.100.23) is a public IP, and the combination of the DNS query "update
1.5	MALICIOUS GENERAL INITIAT{ Bασinos-Agent	MALICIOUS GENERAL INITIAT{ Bασinos-Agent** Some clues lining evidence within the specifics indi	Classified as MALICIOUS Incorrect website queried** Reasoning N.O-YARA draped Believeholder beh	Classification: MALICIOUS. This log entry shows signs of a potential beaconingобщепининаяactivi

TEMPERATURE: CONTROLLING RANDOMNESS



Parameter Interactions

	Low Temp (0.1)	Med Temp (0.7)	High Temp (1.2)
Top-p = 0.1	Most deterministic. Same output every time.	Slightly variable. Still constrained by top-p.	Constrained by top-p. Temp wants chaos, p says no.
Top-p = 0.5	Deterministic. Temp dominates.	Balanced. Good default zone.	Moderate creativity. Some surprises.
Top-p = 0.9	Still focused. Temp dominates.	Good exploration. Recommended range.	Maximum diversity. Risk of incoherence.

Key insight: These parameters interact. The dominant parameter "wins." Low temperature overrides high top-p. Very low top-p constrains high temperature.

Other Inference Parameters

Parameter	What It Controls	Typical Range
max_tokens	Maximum response length	100–4096+
frequency_penalty	Penalizes repeated tokens (proportional to count)	0.0–2.0
presence_penalty	Penalizes tokens that have appeared at all	0.0–2.0
stop sequences	Tokens that halt generation	Custom strings
seed	Reproducibility (deterministic sampling)	Any integer

Parameter Selection Guidelines for Cyber

Task	Temperature	Top-p	Why
Alert classification	0.0–0.2	0.1–0.3	Consistency matters. Same alert → same classification.
IOC extraction	0.0–0.1	0.1	Precision critical. Don't hallucinate indicators.
Threat report writing	0.5–0.7	0.8–0.9	Balance accuracy with readable prose.
Red team brainstorming	0.8–1.0	0.9–1.0	Explore creative attack vectors.
Hypothesis generation	0.7–0.9	0.8–0.9	Want diverse possibilities, not convergence.

The more consequential the decision, the lower the temperature.

IMPROVING LLM PERFORMANCE

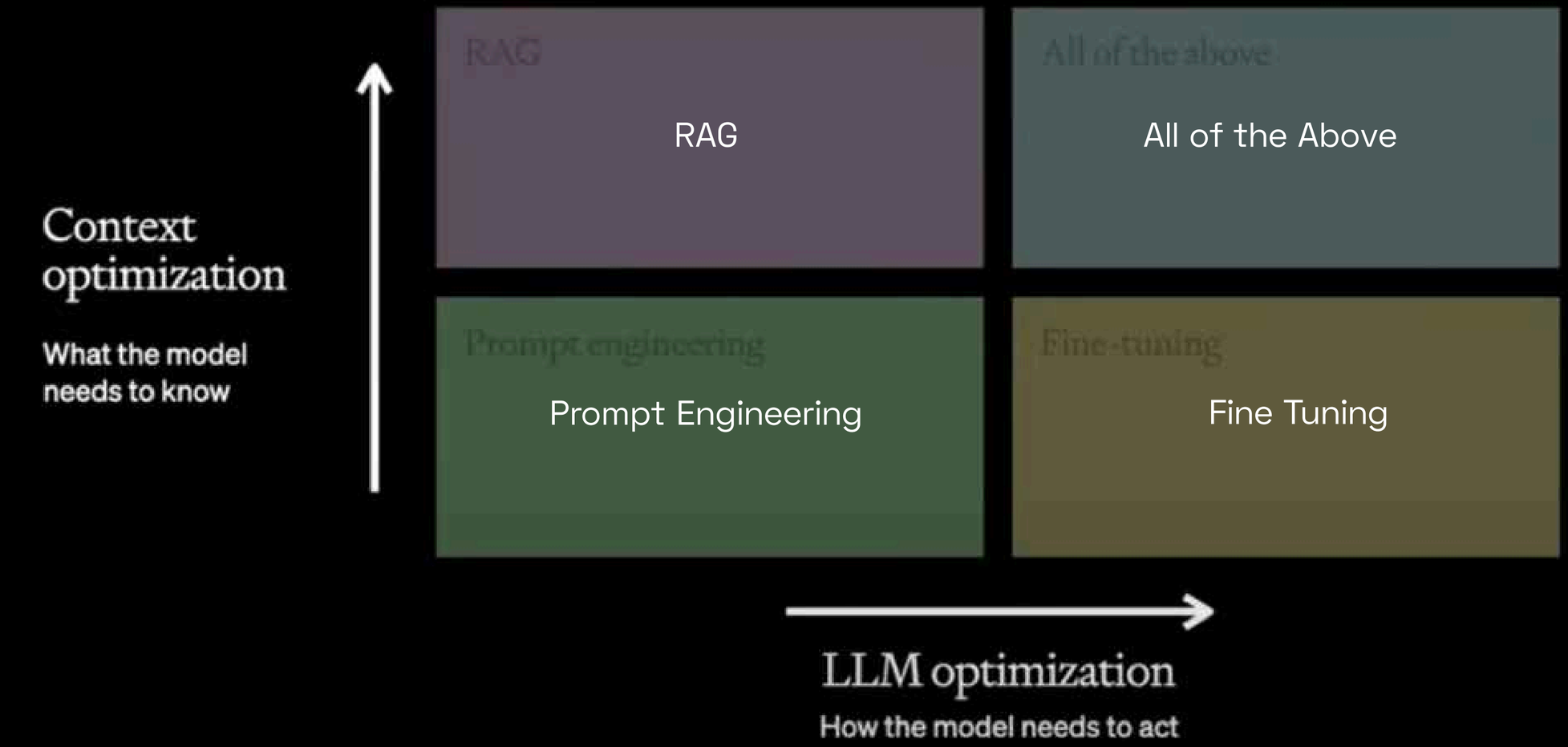
THREE APPROACHES TO BETTER PERFORMANCE

01 Prompt (or Context) Engineering

02 RAG

03 Fine-Tuning

The optimization flow



WHY DOES PROMPTING WORK?

THE GOAL OF PROMPT ENGINEERING IS TO ACTIVATE SPECIFIC DESIRED REGIONS OF THE MODEL'S LEARNED DISTRIBUTION.

QUERY:
What does RBF stand for?

QUERY:
What does RBF stand for?

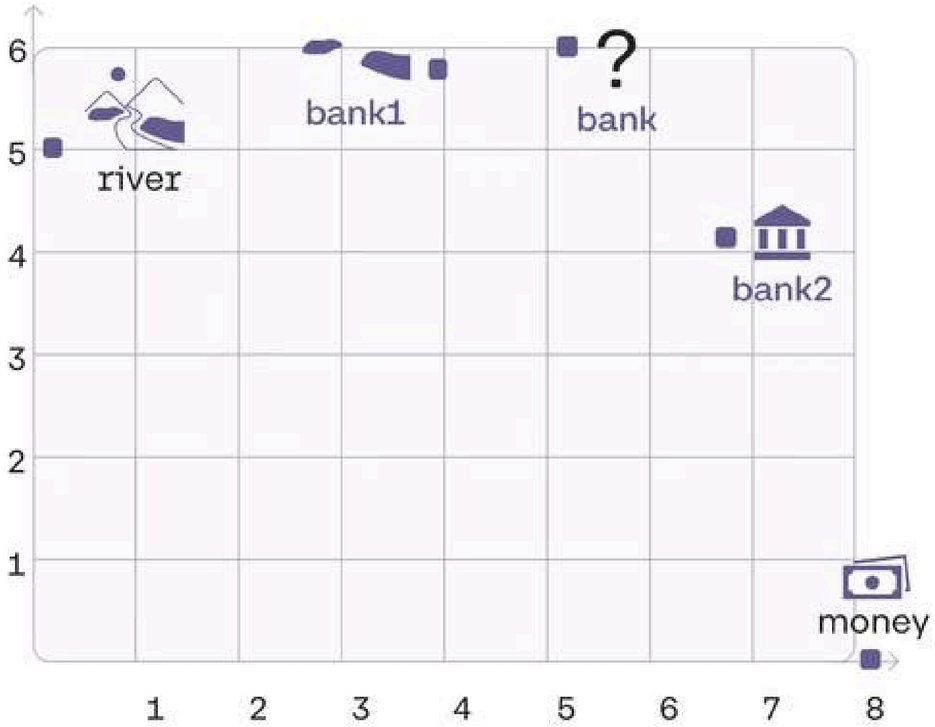
ROLE:
You are a social media influencer.

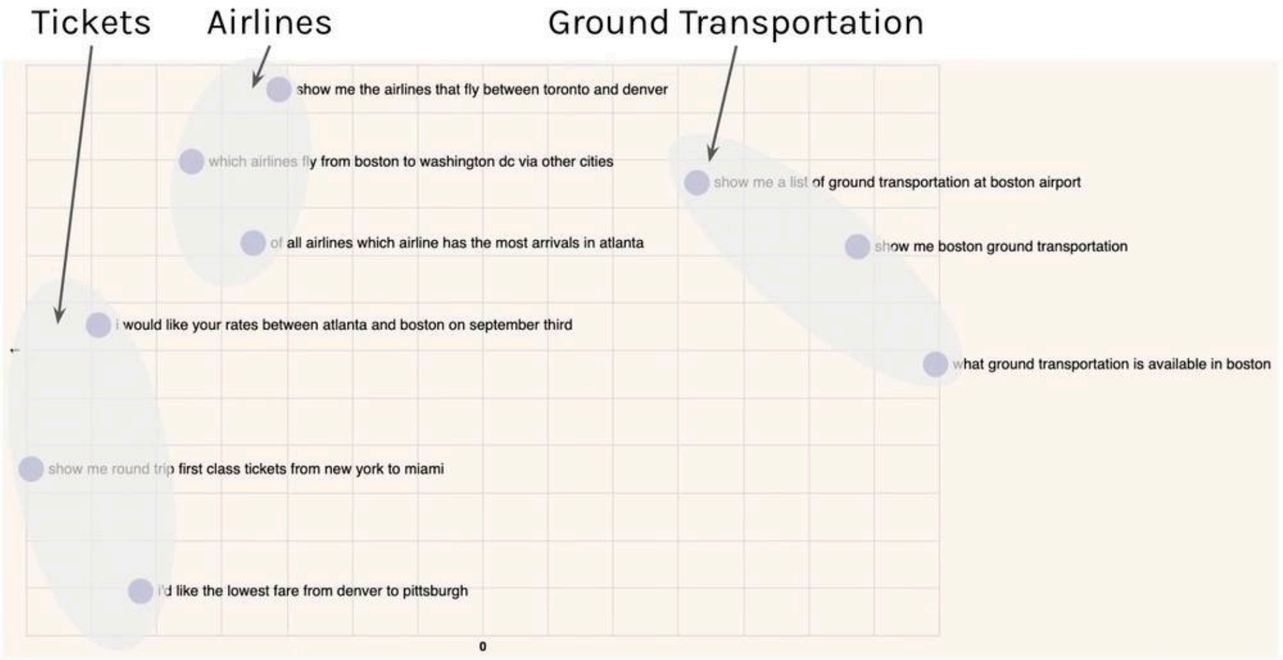
As a social media influencer, I'm here to help! "RBF" typically stands for "Resting ..... " It's a term used to describe a facial expression that may appear uninterested, annoyed, or displeased, even when the person isn't feeling that way. It's important to note that this term can be seen as derogatory or judgmental, as it often criticizes someone's natural facial expression. Always remember to be kind and [...truncated]

ROLE:
You are an expert AI/ML Engineer.

In the context of machine learning and artificial intelligence, **RBF** typically stands for **Radial Basis Function**. Radial Basis Functions are commonly used in various machine learning algorithms, particularly in the context of kernel methods and neural networks.
[...truncated]

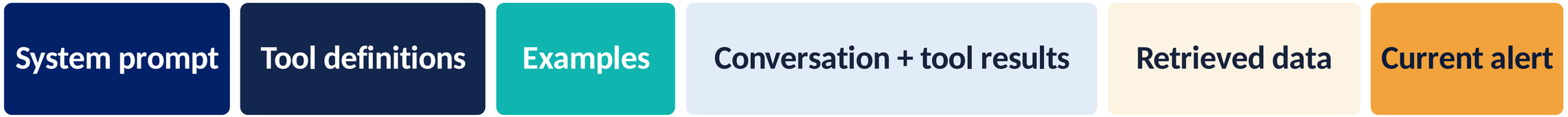
THE PROMPTS HELP GIVE THE MODELS THE PROPER CONTEXT (AND MEANING)





AGENTS EXPAND THE IDEA OF PROMPTS TO INCLUDE ALL CONTEXT THE MODEL SEES

THE CONTEXT WINDOW



MECHANICS 2 OF 3

Context engineering

Prompt engineering asks: how do I phrase the instruction? Context engineering asks the bigger question: at each step, which tokens should be in the window at all?

System prompt at the right altitude

Specific enough to constrain behavior, general enough to survive inputs you did not anticipate. Not a legal contract of edge cases.

Few good examples beat many rules

Three canonical worked examples of a triaged alert teach more than a page of instructions.

Retrieve just in time

Pull the KEV entry or asset record when needed instead of preloading the database. Keep the window high-signal.

Structure the output

Demand a schema: disposition, severity, rationale, evidence. Structured output is easier to validate, log and evaluate.

Compact long histories

Summarize resolved investigation threads instead of dragging full transcripts through every subsequent call.

TRIAGE BOT PREVIEW

Window per alert: the alert JSON, the asset's context, tool results, an output schema. Not the SIEM export.

STRATEGIES FOR BETTER RESULTS

01

BE CLEAR & DIRECT

02

ASSIGN A ROLE

03

USE EXAMPLES

04

USE XML TAGS

05

LET IT THINK

06

CHAIN PROMPTS

Source: docs.anthropic.com/en/docs/build-with-claude/prompt-engineering

BE CLEAR AND DIRECT

✗ Vague

"WHAT DO YOU THINK OF THIS EMAIL?"

✓ Specific

“ANALYZE THIS EMAIL FOR SIGNS OF PHISHING. IDENTIFY ANY POTENTIAL IOCS. CLASSIFY AS BENIGN, SUSPICIOUS, OR MALICIOUS.”

1. BE CLEAR AND DIRECT

DON'T MAKE THE MODEL GUESS YOUR INTENT.

INCLUDE:

- **WHAT FORMAT YOU WANT**
- **WHAT DOMAIN YOU'RE IN**
- **WHAT SUCCESS LOOKS LIKE.**

2. GIVE THE MODEL A ROLE

Role: "You are an expert cybersecurity analyst
with 15 years of SOC experience."

2. GIVE THE MODEL A ROLE

WHATS HAPPENING

Token-level:	Role tokens activate regions of the model trained on that domain's language patterns.
MoE-level:	System prompt tokens influence which experts the router selects for all subsequent tokens. (more on routing later)
Distribution-level:	You're conditioning the probability distribution. i.e. shifting what the model considers 'likely' output.

3. USE EXAMPLES

ZERO-SHOT

No examples. Model relies entirely on training.

FEW-SHOT

2–5 labeled examples. Model learns the pattern from your data.

MANY-SHOT

10+ diverse examples. Includes edge cases (spear phishing that looks legit).

3. USE EXAMPLES

BEST PRACTICES

- Wrap examples in <example> tags (Anthropic best practice)
- Include diverse cases
- Key edge cases: spear phishing that looks legitimate, legitimate emails with urgency language
- The model learns decision boundaries from your examples

4. USE XML TAGS TO STRUCTURE PROMPTS

```
<context>
You are a SOC Tier 2 analyst reviewing escalated alerts.
</context>

<alert_data>
{alert JSON payload}
</alert_data>

<instructions>
Determine if this alert is a true positive or false positive.
Cite specific evidence from the alert data.
</instructions>

<output_format>
Verdict: [TRUE_POSITIVE | FALSE_POSITIVE]
Confidence: [HIGH | MEDIUM | LOW]
Evidence: [specific fields/values]
Recommended Action: [escalate | close | investigate further]
</output_format>
```

When prompts have multiple components, XML tags dramatically improve parsing accuracy.

5. LET IT THINK

BASIC

"Think step-by-step"

Lacks guidance on how.

GUIDED

Specify reasoning steps:
1. Check for urgency
2. Examine URLs
3. Assess sender
4. Final classification

STRUCTURED

Use XML tags:
<thinking>
step-by-step analysis
</thinking>
<answer>
classification + evidence
</answer>

5. LET IT THINK

Warning: Reasoning models handle CoT natively and forced CoT results in worse performance

The Psychology of Prompting

"That's not right, try again"

This induces an apology. Not because the model has internal state — apology-after-correction is a dominant pattern in the training corpus. The model produces what statistically follows that sequence.

The Good Will Hunting Effect

A mentor who believes in someone → better outputs. By prompting with authority and encouragement, you activate that part of the training distribution. The model produces what would follow those conditions in its training data.

The double-edged sword: the same pattern matching that makes LLMs powerful is exactly what makes them exploitable.

6. CHAIN COMPLEX PROMPTS

BREAK ANALYSIS INTO SUBTASKS.

- Each step can use different temperature / top-p settings
- Each step can use a different model (fast for extraction, powerful for assessment)
- Errors are isolated — a bad extraction doesn't corrupt the entire analysis
- Each step can be evaluated and improved independently

Step 1 → input to Step 2.

6. CHAIN COMPLEX PROMPTS

1

Extract IOCs

Parse email for sender, URLs, attachments, headers

→

2

Enrich

Cross-reference IOCs against threat intel feeds

→

3

Assess Risk

Score threat level, map to MITRE, identify campaign

→

4

Recommend

Generate response actions and executive summary

WORKFLOW

LLMS AND TOOLS ARE ORCHESTRATED THROUGH PREDEFINED PATHS.

Your **code** decides the route. The model fills in the steps.

AGENT

LLMS DYNAMICALLY DIRECT THEIR OWN PROCESSES AND TOOL USAGE.

The model decides the route, based on what it observes.

```python
# ──────────────────────────────────────────────
# STEP 1: Extract IOCs (low temperature = precision)
# ──────────────────────────────────────────────
step1 = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": "You are an IOC extraction specialist. Return only valid JSON."},
        {"role": "user", "content": f"""Extract all Indicators of Compromise from this email.
Return as JSON with these keys: urls, domains, ips, email_addresses, urgency_indicators, suspicious_patterns.

Email:
{PHISHING_EMAIL}"""}
    ],
    temperature=0.1,
    max_tokens=500
)
```

STEP 1 — Extracted IOCs:

```json
{
  "urls": [
    "http://boa-secure.xyz/verify?id=8a7f3b"
  ],
  "domains": [
    "bankofamerica-secure-verify.com",
    "boa-secure.xyz"
  ],
  "ips": [],
  "email_addresses": [
    "security@bankofamerica-secure-verify.com"
  ],
  "urgency_indicators": [
    "URGENT",
    "unauthorized access",
    "Failure to verify within 24 hours will result in permanent account suspension"
  ],
  "suspicious_patterns": [
    "Click here to verify your identity",
    "Failure to verify within 24 hours will result in permanent account suspension"
  ]
}
```

```python
# ──────────────────────────────────────────────
# STEP 2: Classify using extracted IOCs (slightly higher temp)
# ──────────────────────────────────────────────
step2 = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": "You are a senior threat intelligence analyst."},
        {"role": "user", "content": f"""Based on these extracted IOCs, classify the original email.

<extracted_iocs>
{iocs}
</extracted_iocs>

<instructions>
1. Classify as BENIGN, SUSPICIOUS, or MALICIOUS
2. Provide confidence level (HIGH/MEDIUM/LOW)
3. Map to MITRE ATT&CK techniques if applicable
4. Recommend SOC action (close, investigate, escalate)
</instructions>"""}
    ],
    temperature=0.2,
    max_tokens=500
)
```

```
### Classification:
**MALICIOUS**

### Confidence Level:
**HIGH**

### Analysis:
1. **Domains and URLs**:
   - The domains "bankofamerica-secure-verify.com" and "boa-secure.xyz" are suspicious. They mimic legitimat
   - The URL "http://boa-secure.xyz/verify?id=8a7f3b" is likely a phishing link designed to capture sensitiv

2. **Email Address**:
   - The email address "security@bankofamerica-secure-verify.com" is suspicious because it uses a domain tha

3. **Urgency Indicators**:
   - Phrases like "URGENT," "unauthorized access," and "Failure to verify within 24 hours will result in per

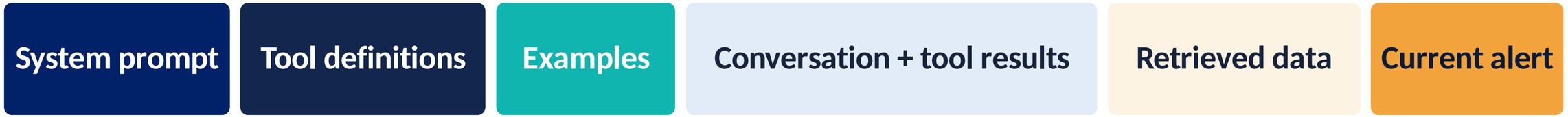
4. **Suspicious Patterns**:
   - The phrase "Click here to verify your identity" is a classic phishing tactic designed to lure the recip
   - The threat of account suspension is another common phishing technique to pressure the recipient into ta

### MITRE ATT&CK Techniques:
- **T1566.002 - Phishing: Spearphishing Link**: The email contains a suspicious link designed to capture sen
- **T1598.003 - Spearphishing Attachment**: Although no attachment is mentioned, the email's content and str
- **T1078.004 - Valid Accounts: Cloud Accounts**: The email aims to trick the recipient into providing crede

### SOC Action:
**ESCALATE**
```

WITH AGENTS YOU NEED TO THINK OF THIS AT EVERY STEP

THE CONTEXT WINDOW



THE CONTEXT WINDOW

It is one sequence

System prompt, tools, history, retrieved docs and the current input all share one fixed token budget. They compete.

Attention is not uniform

Long contexts degrade: material buried mid-window gets used less reliably than material at the edges. More context is not automatically better.

There is no memory

The model is stateless between calls. Whatever your agent 'remembers', your code re-sent it. Memory is an engineering feature you build.

CONTEXT ENGINEERING

- System prompt at the right altitude
- Few good examples beat many rules
- Retrieve just in time
- Structure the output
- Compact long histories

FROM LLMS TO AGENTS

WITH AGENTS:

THE MODELS “GENERATE THEIR OWN SEARCH QUERIES, SELECT APPROPRIATE TOOLS, AND DETERMINE WHAT INFORMATION TO RETAIN.”

Source: Anthropic

LLM

reasoning + generation

RETRIEVAL

Pulls reference data into context: asset inventory, CTI reports, runbooks, past incidents.

TOOLS

Requests actions from the outside world: query the SIEM, look up a CVE, detonate a sample.

MEMORY

Persists state across calls: case notes, investigation history, what has been tried.

ReAct LOOP

THE AGENT LOOP

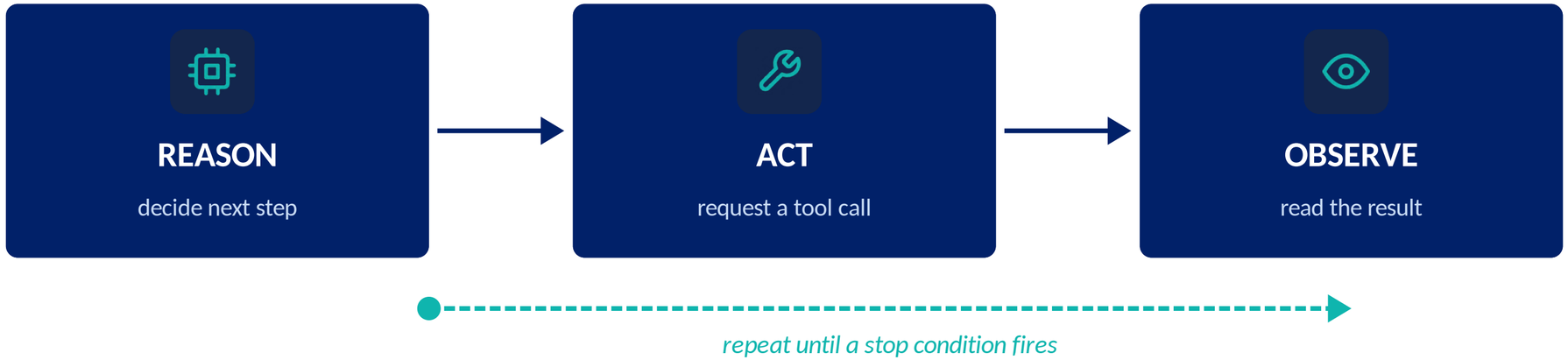
“Agents are typically just LLMs using tools based on environmental feedback in a loop.”

Building Effective Agents

THE AGENT LOOP



THE AGENT LOOP



INTERLEAVED · ReACT

Think between every action: reasoning traces and tool calls alternate. Adapts fast; spends tokens every turn.

PLAN, THEN EXECUTE

Draft the whole plan up front, run the steps, replan on failure. Cheaper and auditable; a stale plan is the risk when the environment shifts.

STOP CONDITIONS:

- Goal met
- Budget spent: max turns, tokens, dollars
- Checkpoint: a human approves before continuing

TASK DECOMPOSITION

Does it break into fixed stages?

Same steps for every input, each stage feeding the next.

PROMPT CHAINING

Does it break into kinds?

Distinct categories, each better handled by its own specialist.

ROUTING

Does it break into independent parts?

Pieces that do not need each other; or one judgment worth several independent opinions.

PARALLELIZATION

Does the break depend on the input?

You only know the subtasks after seeing the specific case.

ORCHESTRATOR-WORKERS

No clean slice? If you can state the quality bar, iterate against it (evaluator-optimizer). If you cannot even fix the path, that is agent territory: buy it with budgets, fences, and an eval. Worksheet boxes 1 and 5 are this slide.

TASK DECOMPOSITION

Does it break into fixed stages?

Same steps for every input, each stage feeding the next.

PROMPT CHAINING

Does it break into kinds?

Distinct categories, each better handled by its own specialist.

ROUTING

Does it break into independent parts?

Pieces that do not need each other; or one judgment worth several independent opinions.

PARALLELIZATION

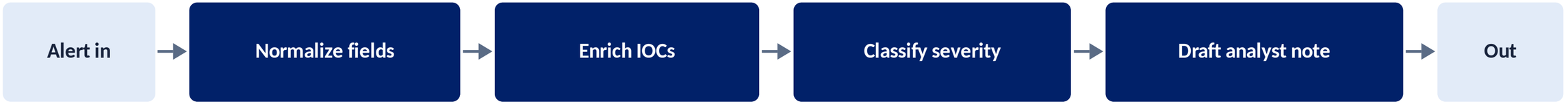
Does the break depend on the input?

You only know the subtasks after seeing the specific case.

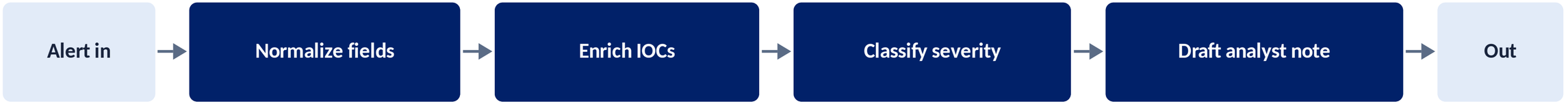
ORCHESTRATOR-WORKERS

No clean slice? If you can state the quality bar, iterate against it (evaluator-optimizer). If you cannot even fix the path, that is agent territory: buy it with budgets, fences, and an eval. Worksheet boxes 1 and 5 are this slide.

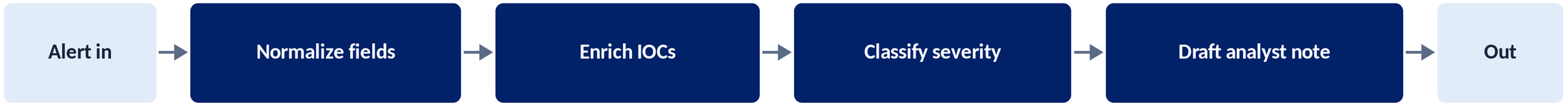
PROMPT CHAINING



PROMPT CHAINING (SEQUENTIAL AGENTS)



PROMPT CHAINING



WHEN TO USE IT

- The task decomposes cleanly into fixed subtasks

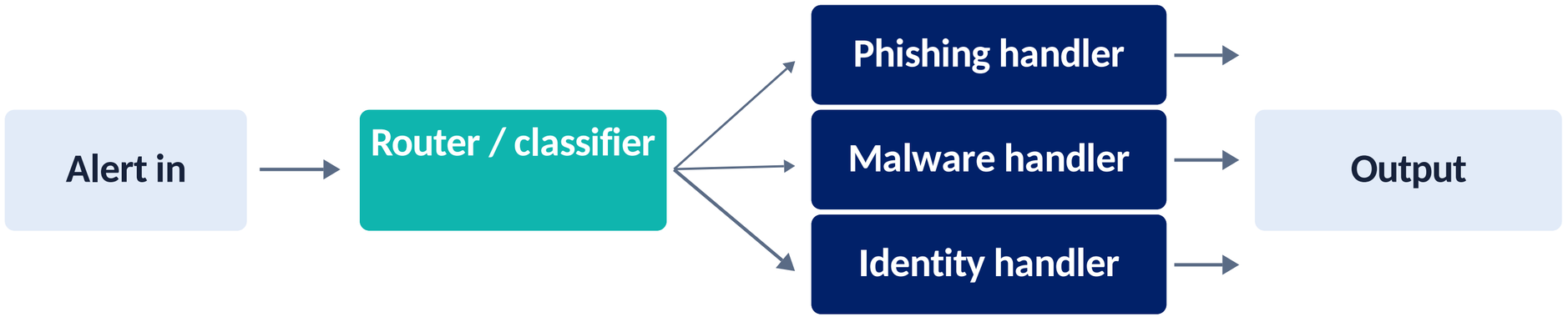
STRENGTHS

- Each step is simple and testable
- Natural checkpoints for gates and logging
- Easy to reason about and debug

RISKS

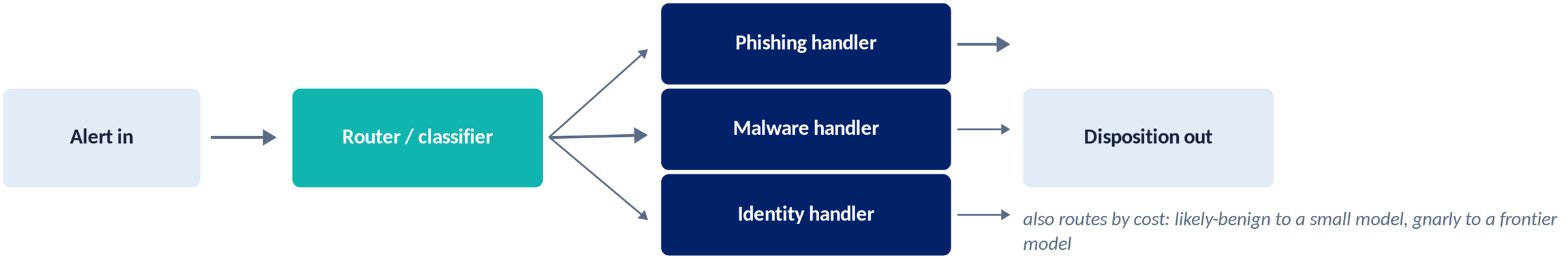
- Latency stacks per step
- An early error propagates downstream
- Rigid: the path cannot adapt to the input

ROUTING



Note: Many systems are now also managing cost.

ROUTING



WHEN TO USE IT

- Inputs fall into distinct categories that are better handled separately, and classification is reliable enough to trust up front.

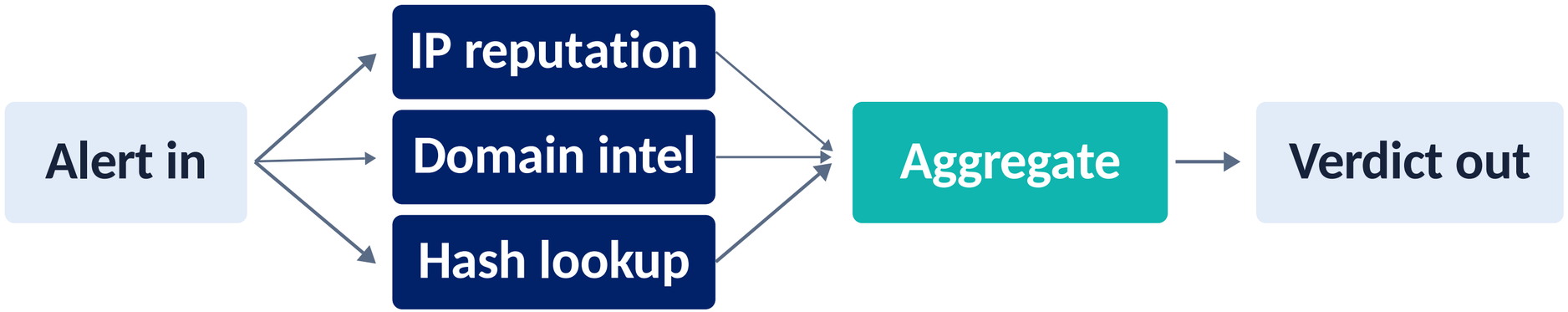
STRENGTHS

- Separation of concerns per category
- Each handler prompt optimized alone
- Cheap wins via model-size routing

RISKS

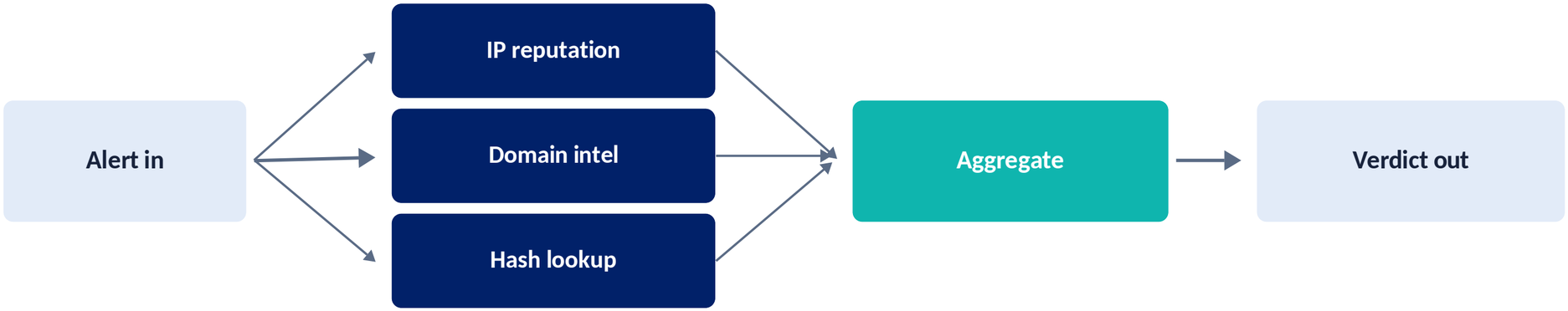
- Router mistakes poison everything after
- Categories drift; taxonomy needs upkeep
- One more component to evaluate

PARALLELIZATION



voting variant: run the SAME question through different models or variations of the same questions, require agreement

PARALLELIZATION



WHEN TO USE IT

- Subtasks are independent, so they can run at once (sectioning), or
- one judgment deserves several independent attempts (voting).

STRENGTHS

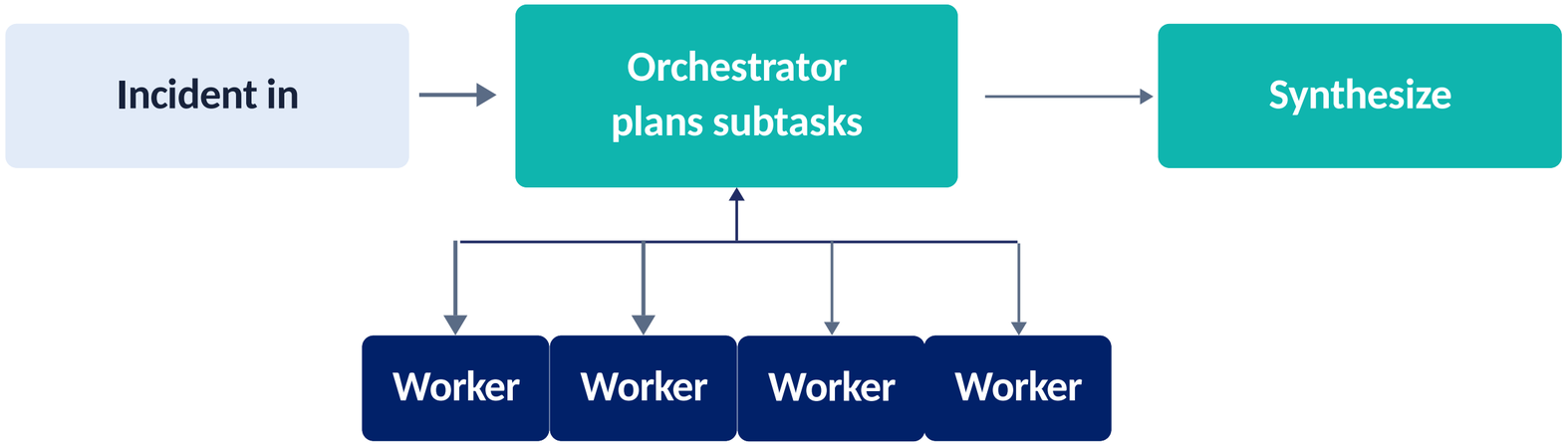
- Latency of the slowest branch, not the sum
- Voting buys calibrated confidence
- A guardrail can run parallel to the worker

RISKS

- Cost multiplies with branches and votes
- Aggregation logic required thoughtful design
- Independent runs can share blind spots

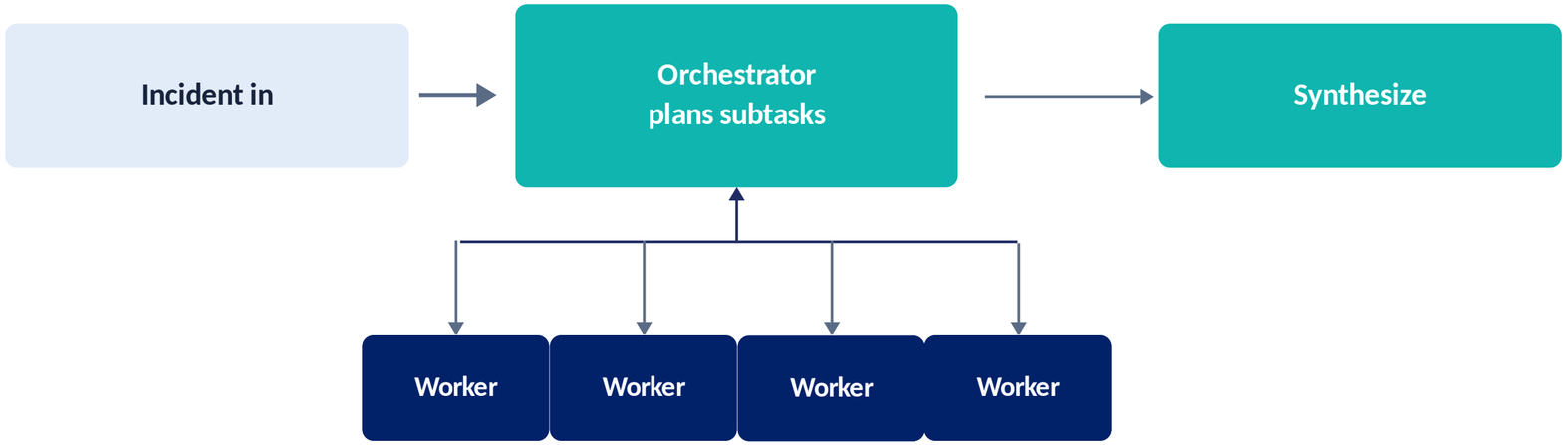
PATTERN 4 OF 5

Orchestrator-workers



PATTERN 4 OF 5

Orchestrator-workers



WHEN TO USE IT

- The task is complex but you cannot predict the subtasks in advance.
- The decomposition itself requires judgment about the specific input.

STRENGTHS

- Adapts the plan to each incident
- Workers run focused, clean contexts
- Scales to sprawling investigations

RISKS

- Orchestrator quality bounds everything
- Cost and latency are input-dependent
- Hardest of the five to evaluate

PATTERN 5 OF 5

Evaluator-optimizer



WHEN IT FITS

You can state clear evaluation criteria, and iterative refinement against them measurably improves the output.

SECOPS EXAMPLE

Detection engineering: one call drafts a Sigma rule, another tests it against sampled logs and known-bad traces, feeding back false-positive hits until the rule meets the bar.

STRENGTHS

- Quality climbs against an explicit bar
- The critic doubles as documentation
- Great fit when tests are automatable

COSTS AND RISKS

- Needs criteria crisp enough to check
- Cost doubles or worse per iteration
- Can loop forever without an exit rule

PATTERN 5 OF 5

Evaluator-optimizer



WHEN IT FITS

You can state clear evaluation criteria, and iterative refinement against them measurably improves the output.

SECOPS EXAMPLE

Detection engineering: one call drafts a Sigma rule, another tests it against sampled logs and known-bad traces, feeding back false-positive hits until the rule meets the bar.

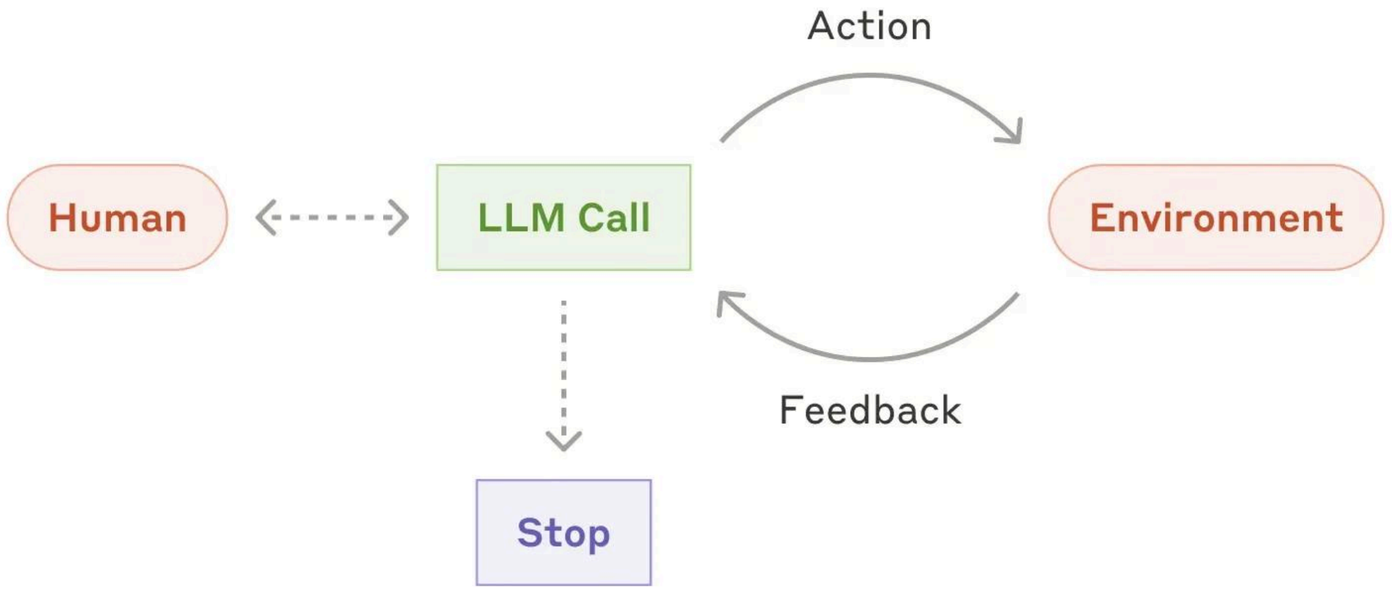
STRENGTHS

- Quality climbs against an explicit bar
- The critic doubles as documentation
- Great fit when tests are automatable

COSTS AND RISKS

- Needs criteria crisp enough to check
- Cost doubles or worse per iteration
- Can loop forever without an exit rule

AUTONOMOUS AGENTS



AUTONOMOUS AGENTS

The model:

- plans
- creates sub-agents
- calls tools
- reads the environment
- adjusts
- Continues until a stop condition fires.

Engineering efforts shifts to guardrails

- Agent sandboxes
- Agent permissions
- tool allowlists
- token budgets
- checkpoints
- etc.

AUTONOMOUS AGENTS

WHEN TO USE IT

- Use when the problem is open-ended and you cannot predict the steps or hardcode a path.

STRENGTHS

- Well suited for complex tasks that require flexibility and adaptability

RISKS

- Higher cost
- Errors that compound across step

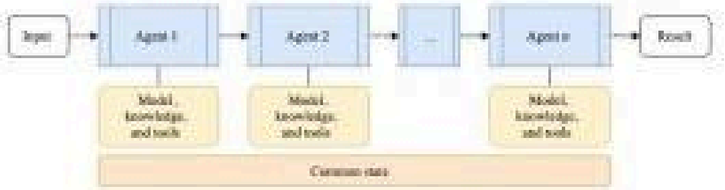
The level of autonomy is a scale and autonomous agents can be combined with any of the other design patterns

CHOOSING A PATTERN

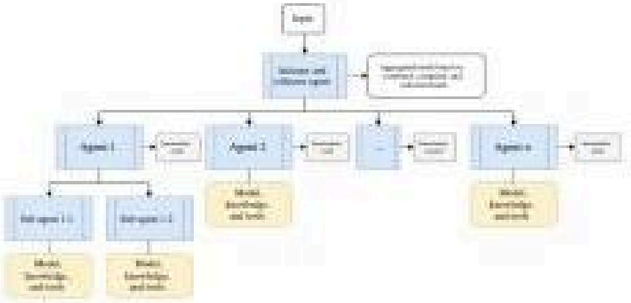
1.	Can one well-fed call with retrieval do it?	→	**Stop there. Keep it simple.**
2.	Does it decompose into fixed, known steps?	→	**Prompt chaining (Sequential Orchestration), with gates.**
3.	Do inputs fall into distinct categories?	→	**Routing.**
4.	Independent subtasks, or need several opinions?	→	**Parallelization: sectioning or voting.**
5.	Subtasks unpredictable until you see the input?	→	**Orchestrator-workers.**
6.	Clear criteria plus value from iteration?	→	**Evaluator-optimizer.**
7.	Open-ended, path cannot be hardcoded at all?	→	**Agent, inside fences, with evals.**

These are not mutually exclusive

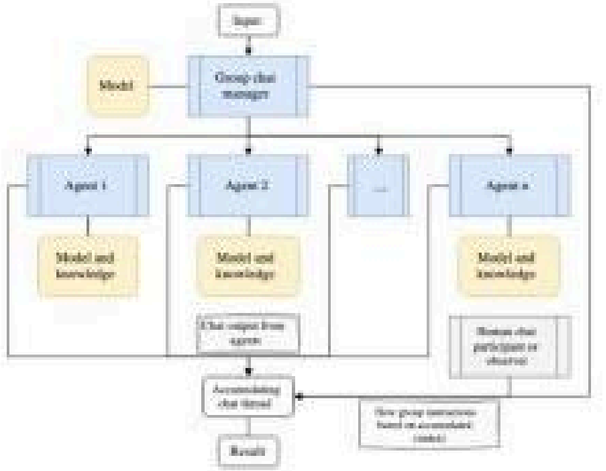
Sequential orchestration



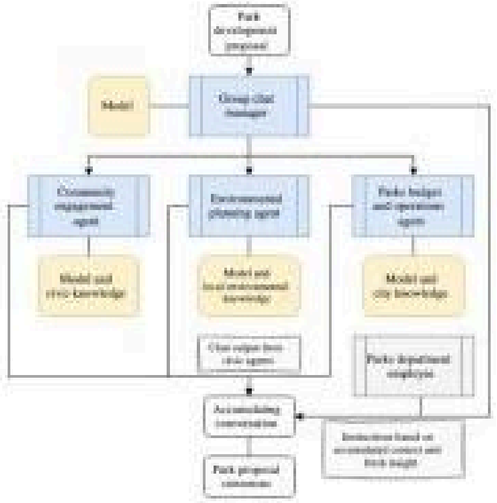
Concurrent orchestration



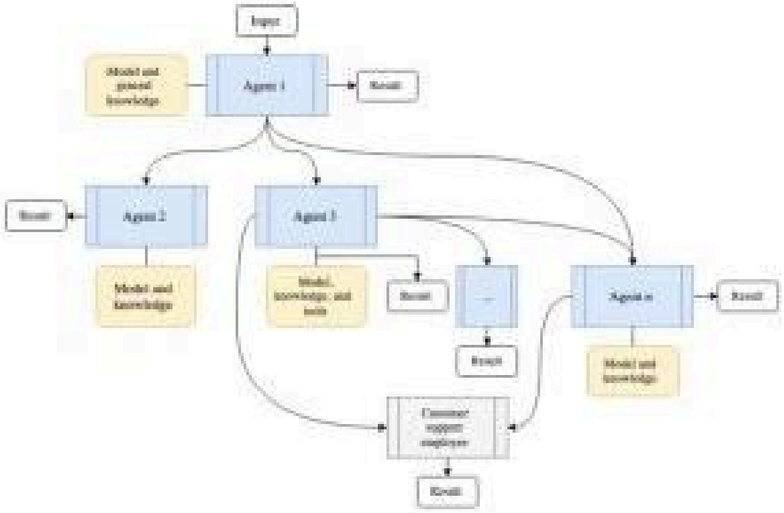
Group chat orchestration



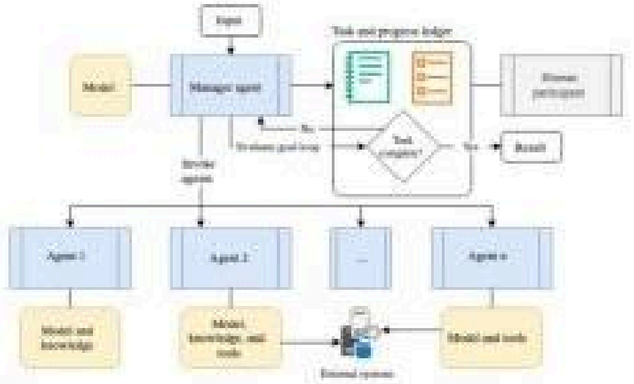
Maker–checker loops

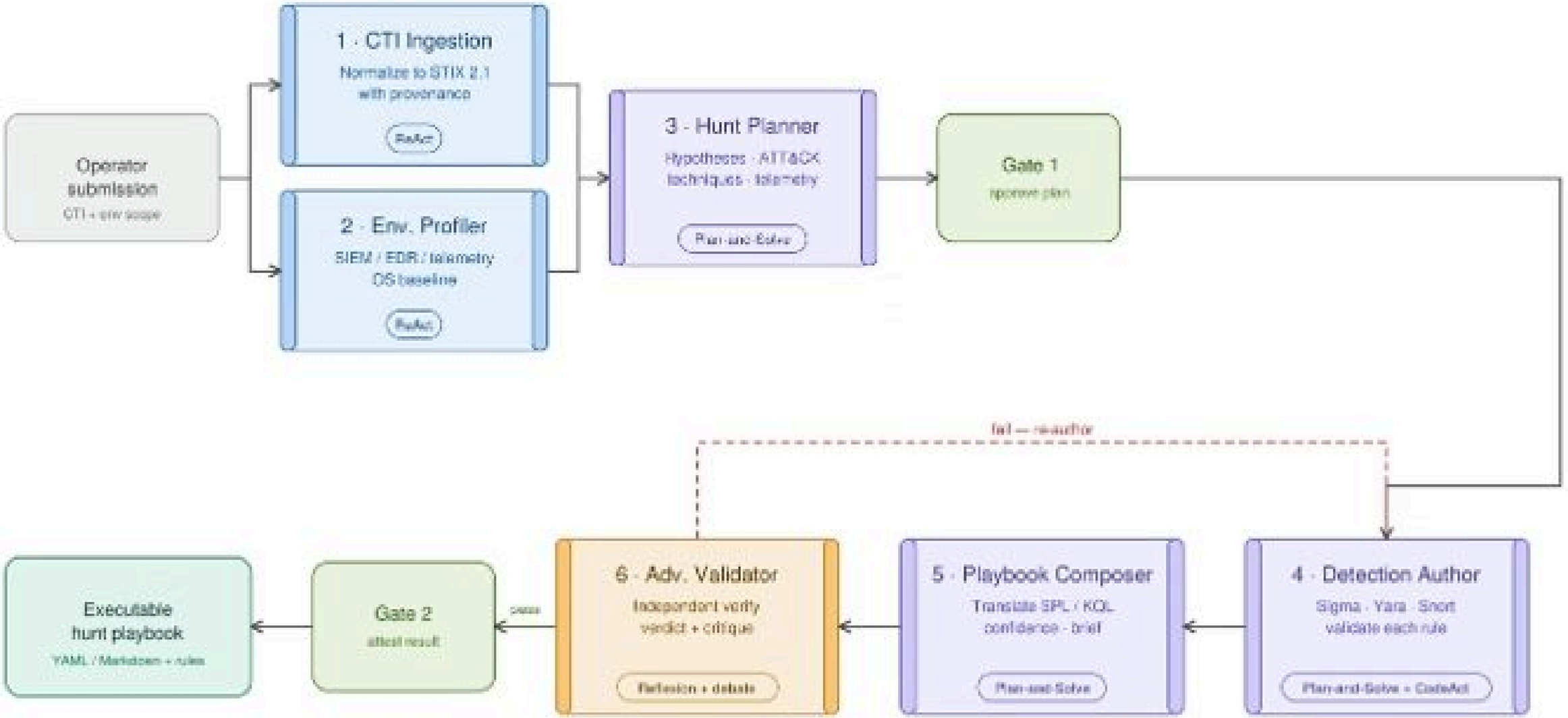


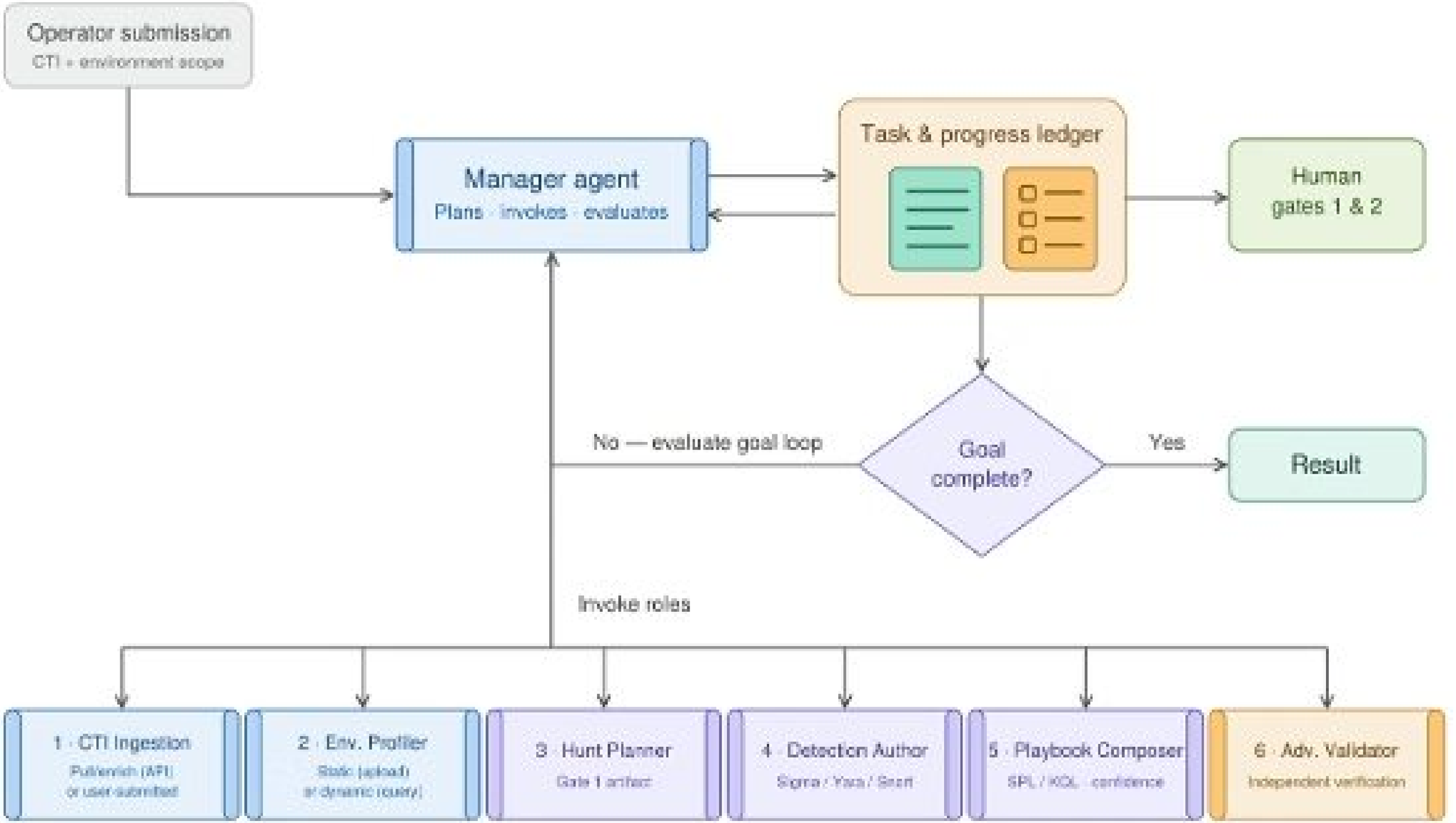
Handoff orchestration



Magentic orchestration




Operator submission
CTI + env scope
1 · CTI Ingestion
Normalize to STIX 2.1 with provenance
ReAct
2 · Env. Profiler
SIEM / EDR / telemetry OS baseline
ReAct
3 · Hunt Planner
Hypotheses · ATT&CK techniques · telemetry
Plan-and-Solve
Gate 1
approve plan
fail — re-author
4 · Detection Author
Sigma · Yara · Snort validate each rule
Plan-and-Solve + CodeAct
5 · Playbook Composer
Translate SPL / KQL confidence · brief
Plan-and-Solve
6 · Adv. Validator
Independent verify verdict + critique
Reflexion + debate
Gate 2
attest result
Executable hunt playbook
YAML / Markdown + rules


Operator submission
CTI + environment scope
Manager agent
Plans · invokes · evaluates
Task & progress ledger
Human
gates 1 & 2
No — evaluate goal loop
Goal
complete?
Yes
Result
Invoke roles
1 · CTI Ingestion
Pull/enrich (API)
or user-submitted
2 · Env. Profiler
Static (upload)
or dynamic (query)
3 · Hunt Planner
Gate 1 artifact
4 · Detection Author
Sigma / Yara / Snort
5 · Playbook Composer
SPL / KQL · confidence
6 · Adv. Validator
Independent verification

PATTERN SUMMARY

Pattern	Use when	SecOps example	Watch out for
Prompt chaining	Fixed, known decomposition	Normalize → enrich → classify → draft	Error propagation; latency stack
Routing	Distinct input categories	Alert type → specialized handler	Router errors cap the whole system
Parallelization	Independent parts, or need multiple opinions	Parallel IOC lookups; 3-vote suppression	Cost multiplies; correlated blind spots
Orchestrator-workers	Subtasks unknowable up front	Incident scoping across hosts and logs	Hard to evaluate; unbounded cost
Evaluator-optimizer	Clear criteria, iteration helps	Sigma rule drafting vs test logs	Vague criteria launder confidence
Autonomous agent	Open-ended, path cannot be hardcoded	Threat hunt in unfamiliar environment	Compounding errors; needs fences + evals

TOOL USE

WHAT IS A TOOL?

- A tool is a capability your code exposes, described by
  - Tool Name
  - Description
  - Parameter schema.

- Anything can be a tool:
  - a SIEM query,
  - a KEV lookup,
  - a sandbox detonation,
  - a ticket update.

- If code can do it, an agent can request it.

TOOL DEFINITION

```
name:  kev_lookup
description:  Return the CISA KEV
catalog entry for a CVE ID, or
'not_listed'. Use this to confirm
known exploitation before raising
severity. Returns JSON with
dateAdded, requiredAction, dueDate.
input_schema:
  { "cve_id": string, required,
    pattern: "CVE-\d{4}-\d{4,7}" }
```

THESE CAN BE ADDED IN SYSTEM PROMPTS

```
system_prompt = """You are a vulnerability triage specialist
working in a mid-size enterprise SOC.

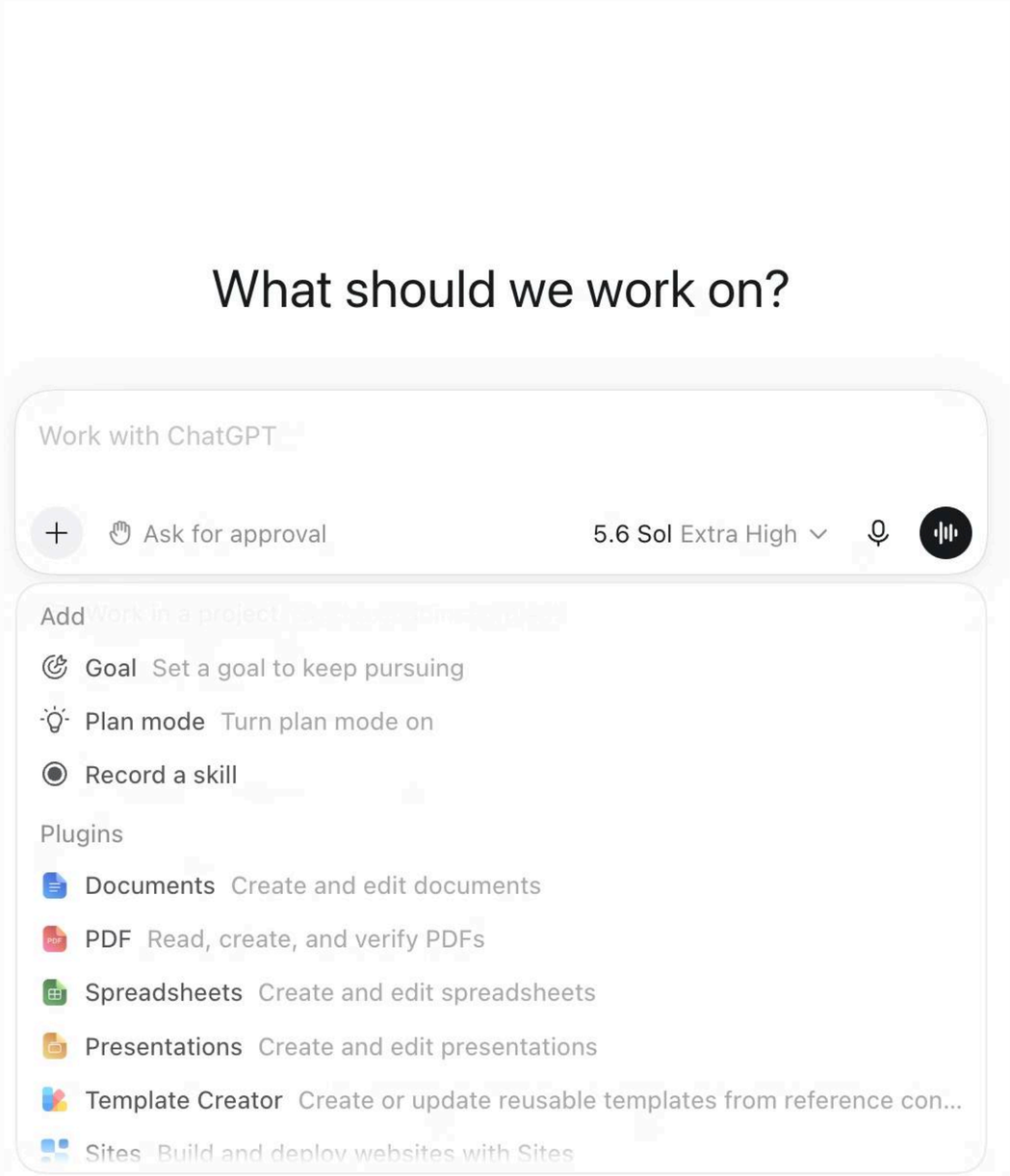
## Role
Analyze CVEs reported by the security team. Gather data
from authoritative sources, assess severity and real-world
impact, and produce actionable risk summaries.

## Tools
You have 3 tools:
- lookup_cve: ALWAYS call this first for any CVE query.
  Returns CVSS score, description, and affected products.
- check_cisa_kev: Call after lookup_cve to check if the
  vulnerability is actively exploited.
- search_threats: Use for additional context on exploitation
  campaigns, threat actor activity, or vendor advisories.

## Output Format
Respond with a structured summary:
- Severity: [CRITICAL/HIGH/MEDIUM/LOW] (CVSS score)
- Exploit Status: [Actively Exploited / No Known Exploits]
- Affected Products: [list]
- Threat Context: [1-2 sentences from search results]
- Recommended Action: [specific remediation steps]

## Guardrails
- NEVER speculate about CVEs not found in NVD.
  If lookup_cve returns "not found", say so explicitly.
- NEVER provide exploitation guidance or PoC code.
- If a tool returns an error, report the error — do not guess.
- Do not call tools that are not listed above.
"""
```

THESE CAN BE ENABLED IN CHAT INTERFACES



THE DOCSTRING IS THE INSTRUCTION MANUAL

VAGUE · UNPREDICTABLE

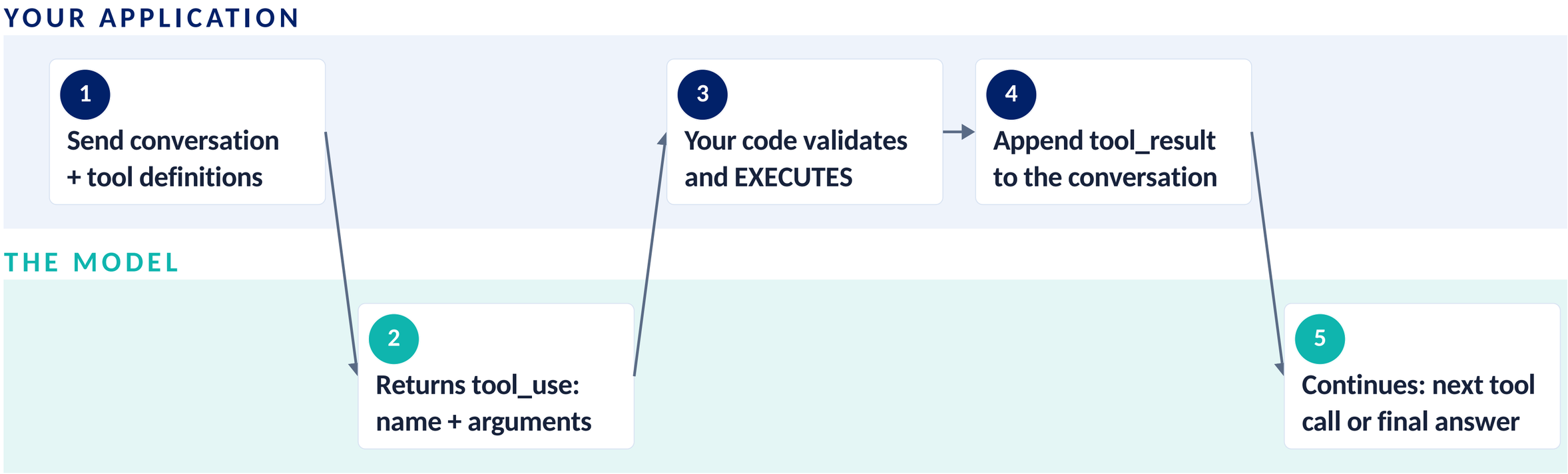
```
@tool
def lookup_cve(cve_id):
    """
    Look up a CVE
    """
```

SPECIFIC · PREDICTABLE

```
@tool
def lookup_cve(cve_id: str) -> str:
    """Search local NVD database for
    CVSS score, description, affected
    products by CVE ID.
    Args:
      cve_id: str, e.g. CVE-2021-44228
    Returns:
      dict with severity, description,
      affected_software"""
```

Describe what the tool does. Include Args and Returns, when to use it, and include a real example value

HOW A TOOL CALL WORKS



WRITING TOOL INTERFACES

Tool definitions deserve as much design attention as human interfaces get.

Write for a junior teammate

The description should let someone who has never seen your system use the tool correctly: purpose, when to use it, what comes back.

Show example calls and edge cases

Include usage examples, boundary conditions, and what distinguishes this tool from its neighbors.

Poka-yoke: make misuse hard

Enums instead of free strings, required fields, absolute paths, validation that rejects garbage before it executes.

Watch the model use it

Run realistic tasks, read the transcripts. When the model fumbles a tool, fix the tool

FAILURE MODES

Hallucinated tools or args

Requests a tool that does not exist, or invents parameter values.

Mitigate: Validate every call against the schema.

Wrong tool for the job

Plausible but wrong choice among overlapping tools.

Mitigate: Sharper descriptions; fewer, more distinct tools.

Over-calling

Loops on a failing tool, burns budget, retries forever.

Mitigate: Max turns, per-run budgets, backoff on repeated failure.

Misreading results

Confidently synthesizes a conclusion the data does not support.

Mitigate: Require verdicts to cite tool output; check citations in evals.

Compounding errors

Small early mistake steers later step off course.

Mitigate: Checkpoints and gates between phases; keep chains short.

Poisoned inputs

Tool results and retrieved text are untrusted input in the window.

Mitigate: Treat output as data, not instructions. Full treatment: W3, W6.

NEXT WEEK

ASSIGNED READINGS

- Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models* (ICLR 2023)
  This is the foundational paper behind the ReAct Loop

- Anthropic, *Effective Context Engineering for AI Agents.*
  Explains the shift in the industry from prompt engineering to context engineering. It explores strategies for effectively curating and managing context.



BACKUP